Introduction to Language Models

Mausam
(Some slides by Tanmoy Chakraborty)

What is an LM?

What is a Language Model?

- Assume a finite vocabulary V of words (or tokens)
  - E.g.: {the, a, student, students, open, opened, exam, exams, his, her, their,.. BOS, EOS}

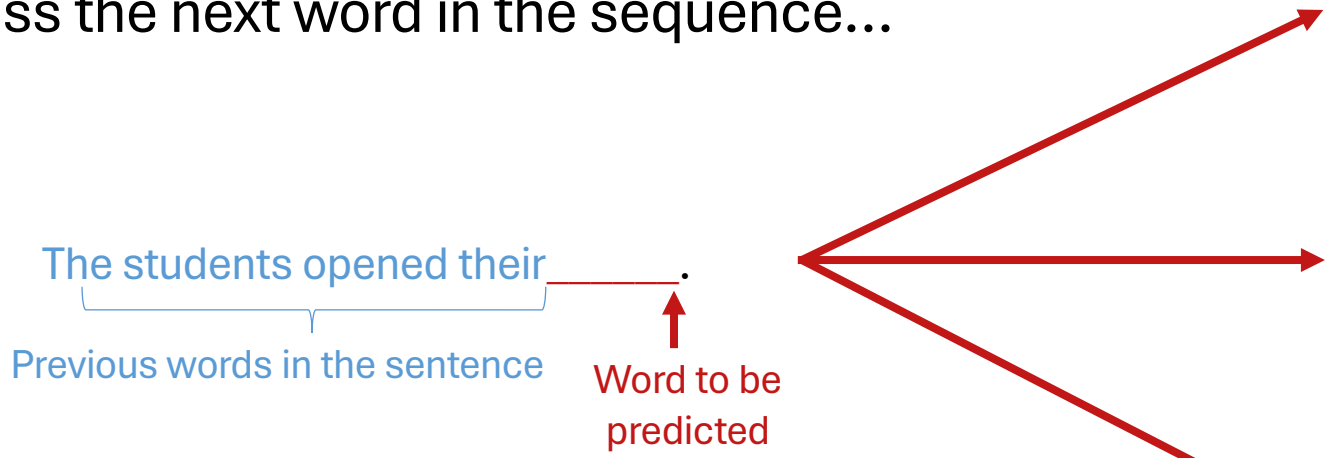
- Intention
  - Generate well-formed (grammatical, meaningful) sentences
  - Assess a sentence for their well-formedness

- Probabilistic Language Model

P_θ(B the E) = 10^{-14}
P_θ(B a E) = $4x10^{-15}$
P_θ(B the students E) = 10^{-15}
P_θ(B the students opened exams E) = 10^{-11}
P_θ(B a students students E) = 10^{-20}
...

Next Word (Token) Prediction

Guess the next word in the sequence...



exams

P_θ(exams | The students opened their)

companies

P_θ(companies | The students opened their)

trees

P_θ(trees | The students opened their)

P_θ(exams|The students opened their) > P_θ(companies|The students opened their) >> P_θ(trees|The students opened their)

Probabilistic Language Models

- **Goal:** Calculate the probability of a sentence or sequence consisting of *n* words

$$P(W) = P(w_1, w_2, w_3, \dots, w_k)$$

or

- **Related Task:** Calculate the probability of the next word conditioned on the preceding words

$$P(w_6 \mid w_1, w_2, w_3, w_4, w_5)$$

A model that calculates **either** of these is referred to as a **Language Model (LM).**

Equivalence: $P(w_1, w_2, w_3, \dots, w_k) = P(w_1)\, P(w_2 \mid w_1)\, P(w_3 \mid w_1\, w_2) \dots P(w_n \mid w_1\, w_{2..}\, w_{k-1})$

Estimate Conditional Probabilities

$$P(\text{exams}|\ \text{The students opened their}) = \frac{\text{Count (The students opened their exams)}}{\text{Count (The students opened their)}}$$

- **Problem:** Enough data is not available to get an accurate estimate of the above quantities.
- **Solution:** Markov Assumption

N-Gram Language Models

Markov Assumption

Every next state depends only the previous *k* states

- Chain Rule:

$$P(w_1 w_2 \ldots w_k) = \prod_i P(wi \mid w_1 w_2 \ \ldots w_{i-1})$$

- Applying Markov Assumption we condition on only the preceding *n* words:

$$P(w_1 w_2 \ldots w_k) = \prod_i P(w_i \mid wi_{-n} \ \ldots w_{i-1})$$

- Probabilistic Language Models exploit the **Chain Rule of Probability** and **Markov Assumption** to build a probability distribution over sequences of words.

N-gram Language Models

- Let's consider the following conditional probability:

  P(exams | The students opened their)

- An **N-gram model** considers only the preceding **N −1 words**.
  - Unigram (0 preceding words): P(exams)
  - Bigram (1): P(exams | their)
  - Trigram (2): P(exams | opened their)

Relation between Markov model and Language Model:

An N-gram Language Model ≡ (N −1) order Markov Model

Limitation of N-gram Language Models

- For a large N
  - Not possible to find data to estimate probabilities (sparsity)

- For a small N
  - High Approximation: Language model quality not very good

Limitation of N-gram Language Models

- An insufficient model of language since they are **not effective in capturing long-range dependencies present in language**.

  - Example:

The **project**, which he had been working on for months, was finally **approved** by the committee.

The example highlights the long-distance dependency between “project” and “approved”,

The context provided by far away words affects the interpretation of later parts of the sentence.

Solution to Fix N-Gram Language Models

- Increase N without increasing sparsity!
- Ideal: make N=all preceding words

- How?
  - Deep Learning: We will come back to it.

Evaluation of Language Models

Evaluation of a Language Model

- Does our language model prefer good sentences to bad ones?
  - Assign higher probability to “real” or “frequently observed” sentences than “ungrammatical” or “rarely observed” sentences

- Terminologies:
  - We optimize the parameters of our model based on data from a **training set**.
  - We assess the model's performance on unseen **test data**
  - **Evaluation metric:** provides a measure of performance of an LM on the test set.

Perplexity

Assign higher probability to sentences seen in the test set!

For the sentence W, perplexity is: $PP(W) = P_\theta(w_1 w_2 \ldots w_k)^{-\frac{1}{k}} = \sqrt[k]{\frac{1}{P_\theta(w_1 w_2 \ldots w_k)}}$

Applying Chain Rule: $PP(W) = \sqrt[k]{\prod \frac{1}{P_\theta(w_i \mid w_1 w_2 \ldots w_{i-1})}}$

For N-Gram (e.g., trigram LMs): $= \sqrt[k]{\prod \frac{1}{P_\theta(w_i \mid w_{i-2} w_{i-1})}}$

Minimizing perplexity is the ~same as maximizing probability.

Why use Perplexity and Not Probability

- Raw Probabilities underflow
- Raw Probabilities have Length Bias

- Perplexity has an intuitive meaning
  - Task of recognizing digits {0,1,…9}: Perplexity = 10
  - Recognizing one of 30000 names at a company: Perplexity = 30000
  - Suppose: 25% operator, 25% sales, 25% tech support, 25% specific person
    - Perplexity = 53

- Perplexity is the weighted equivalent branching factor of next decision

Representation Learning

Solution to Fix N-Gram Language Models

- Increase N without increasing sparsity!
- Ideal: make N=all preceding words

- How? Deep Learning
- Goal #1: create a representation with low sparsity
- Goal #2: build an architecture that can use this information.

Goal #1: create a representation with low sparsity

- What does this even mean?

$$P(\text{exams}|\ \text{The students opened their}) = \frac{\text{Count (The students opened their exams)}}{\text{Count (The students opened their)}}$$

The	students	opened	their
Those/These	people/boys/…	closed/threw/…	

article	noun	verb	pronoun
specific	plural	past	plural
…	animate	phys-action	…
	….	…	

vectors

Word-Property Assignment

- Which properties do we consider?
  - Too many properties to list manually
- How do we assign each word to each property
  - Too many words to do this exercise manually

Solution:

- Properties are induced by data using machine learning
- Assignment of words to properties are also induced by data using ML
- This is called representation learning
  - learning a representation (vector) for each object (word)
- Distributed representation of word meaning

Word2Vec (2014)

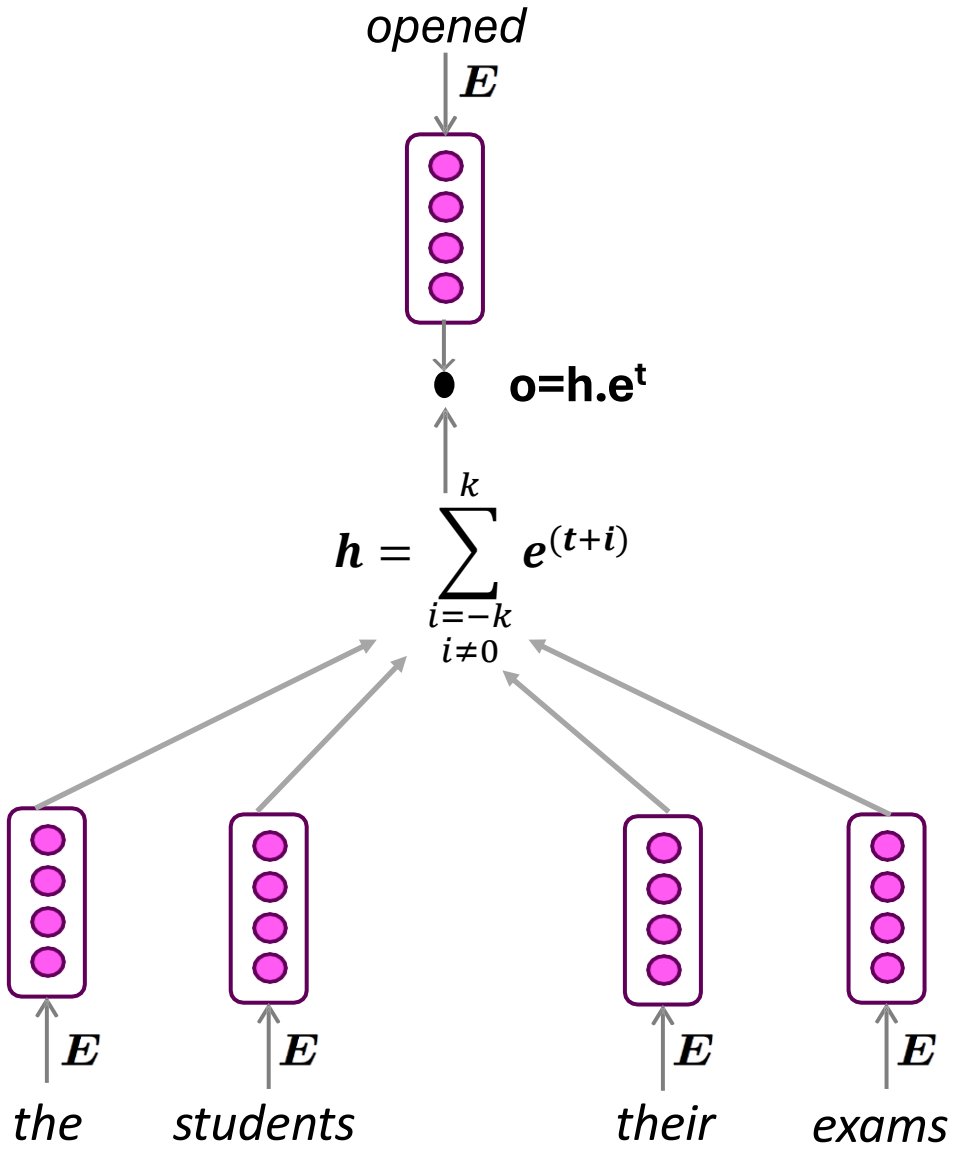
- Goal: Learn a vector for each word so that
  - Similar words have similar vectors
- Representational Choice (dense representation)
  - Each vector is a vector of real numbers in a D-dimensional space
  - Also called word embeddings
- How to find?
  - Distributional Hypothesis: two objects appearing in similar contexts are similar
  - (Firth'57): you know a word by the company it keeps
- Find a model such that: context can help identify the word
  - Find a model that context and word embeddings are similar in latent space

Option 1 (CBOW):

- "as the proctor started the clock, the students opened their exams…"

- Embedding of middle word (w) ~ Overall embedding of context
- Context c= {the, students, their, exams}

- How to represent set
  - addition (cont. bag of words)
- How to represent similarity
  - dot prod.

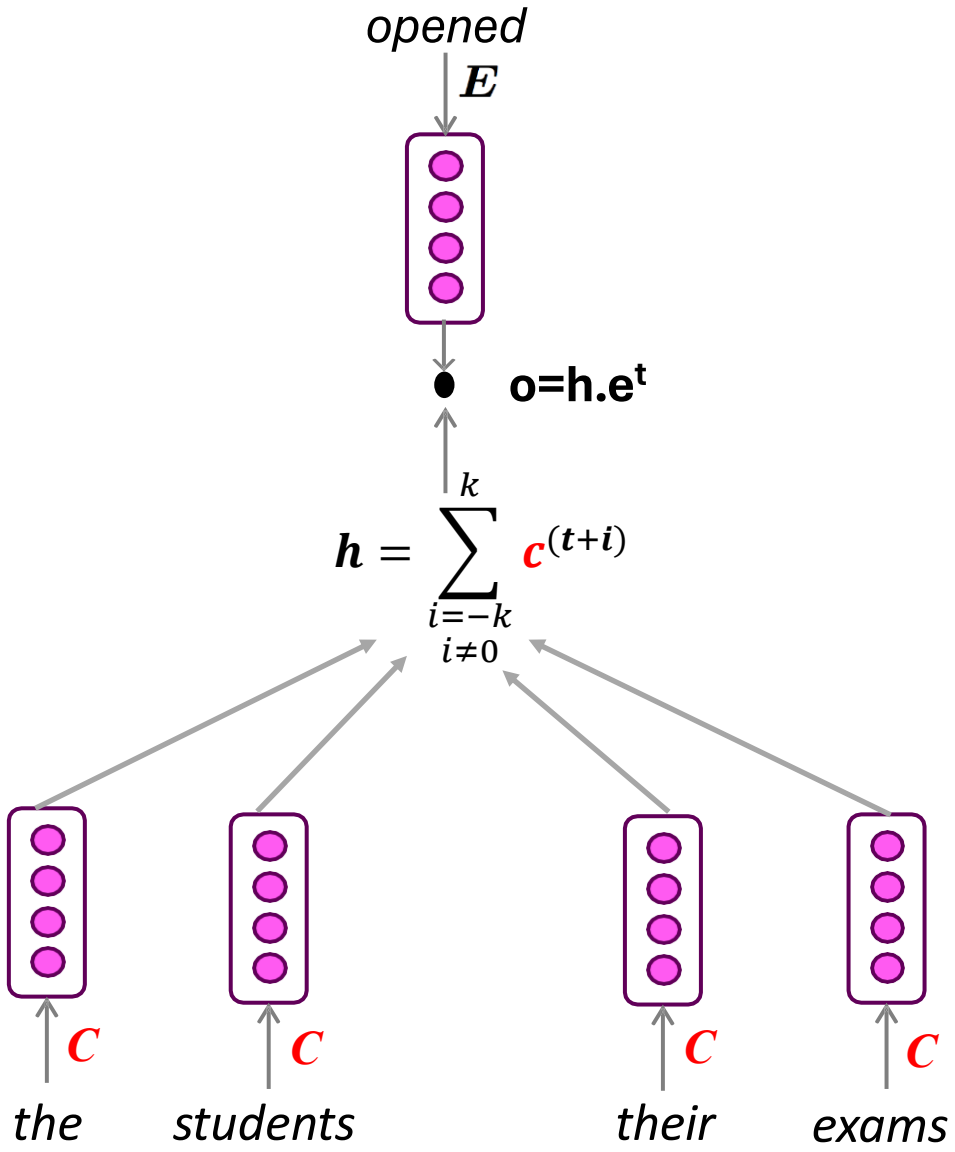


Option 1 (CBOW):

- "as the proctor started the clock, the students opened their exams…"

- Embedding of middle word (w) ~ Overall embedding of context
- Context c= {the, students, their, exams}

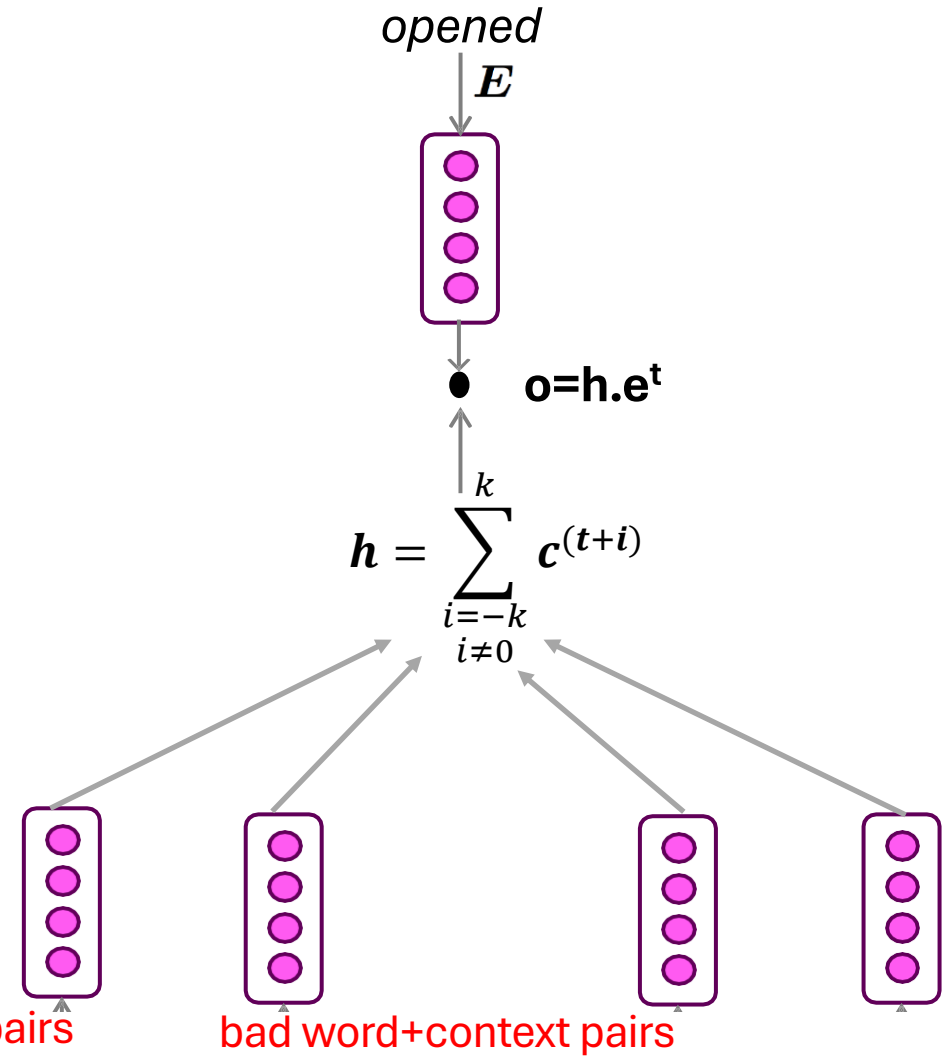
- How to represent set
  - addition (cont. bag of words)
- How to represent similarity
  - dot prod.



Option 1 (CBOW):

- Train embeddings ***E*** such that
- o value is high for “opened”
- o value is low for other random words
  - negative sampling

- Objective Function
  - D = 1: good (w, c) pair
  - Convert o into probability
  - Pr(D=1|c) = σ(o) = σ(h.e^t)
  - Interestingly: 1-σ(o) = σ(-o)



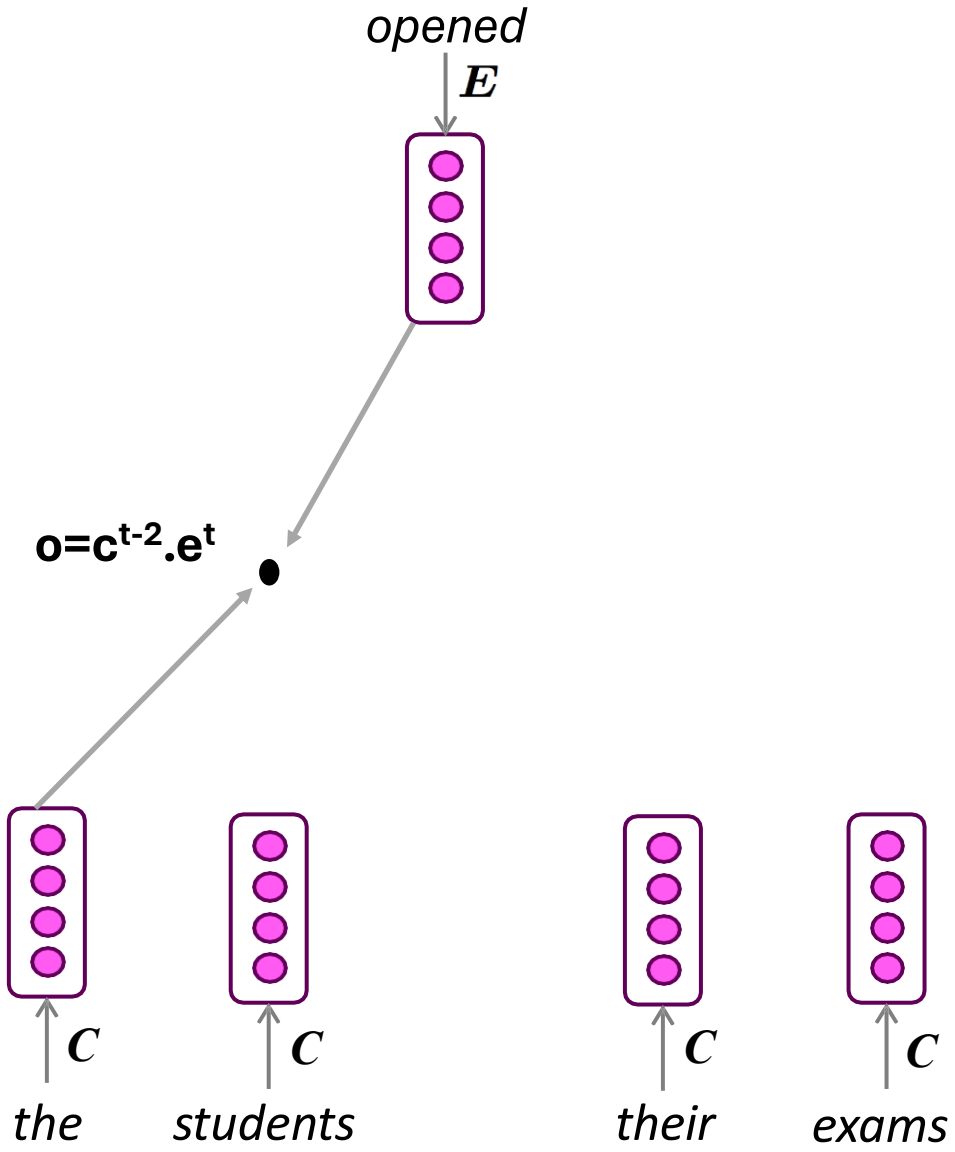
good word+context pairs

bad word+context pairs

$$\mathcal{L}(\Theta; D, \bar{D}) = \sum_{(w,c)\in D} \log P(D=1|w,c) + \sum_{(w',c)\in \bar{D}} \log P(D=0|w',c)$$

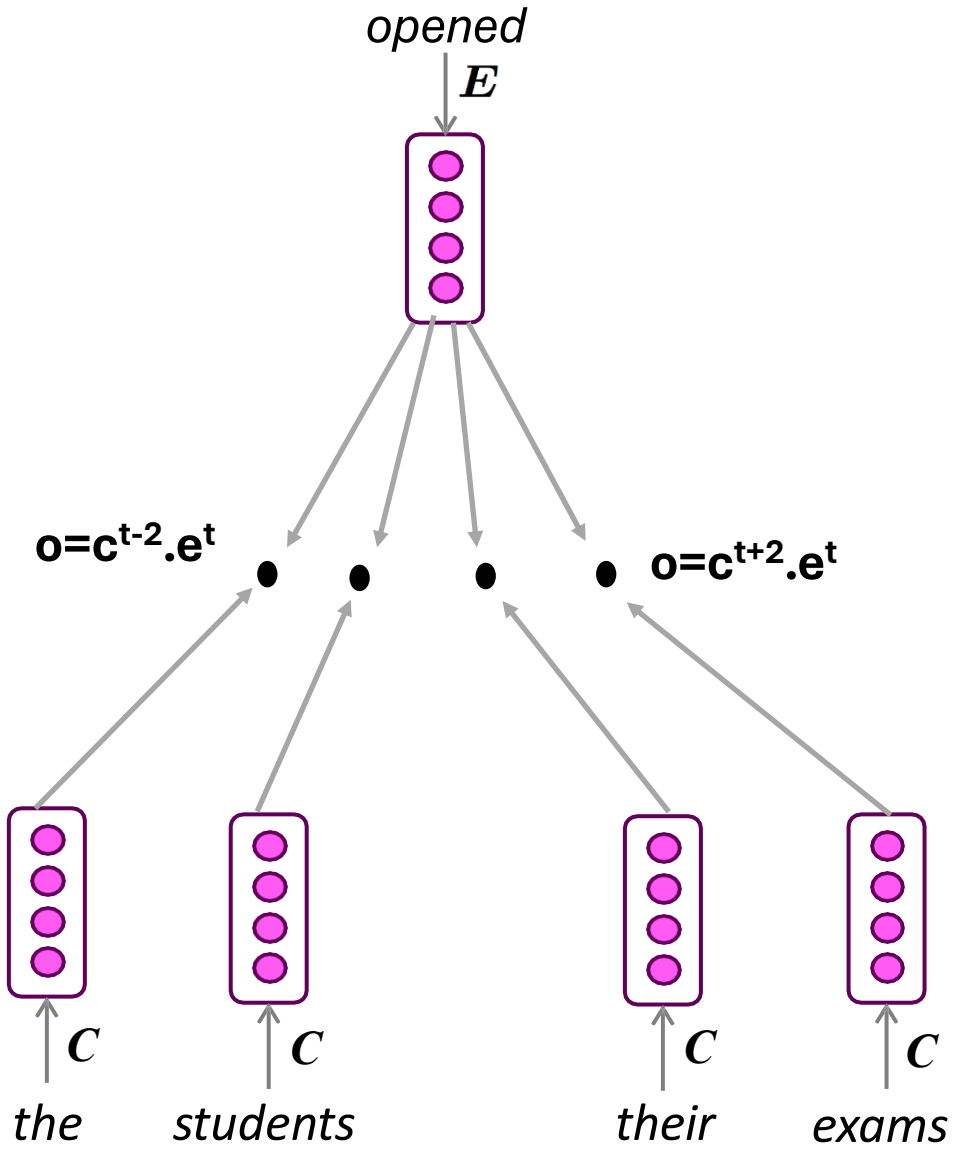
Option 2 (SGNS):

- “as the proctor started the clock, the students opened their exams…”
- Embedding of middle word (w) ~ embedding of each context word



Option 2 (SGNS):

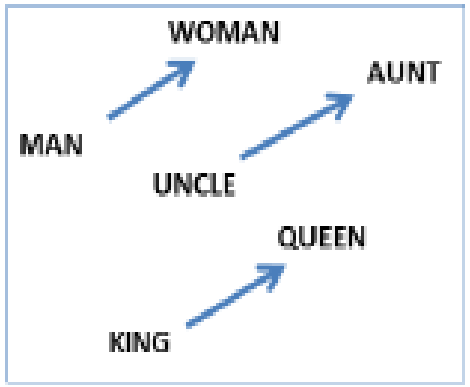
- "as the proctor started the clock, the students opened their exams…"
- Embedding of middle word (w) ~ embedding of each context word
- Skip Gram w/ neg. sampling

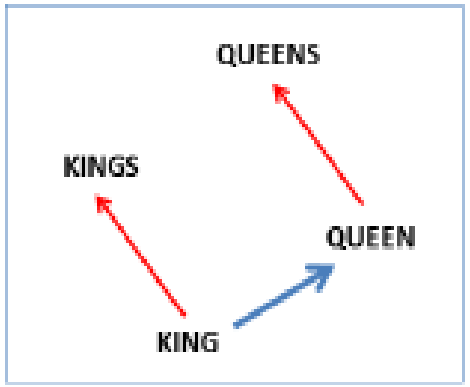


Summary: predict if candidate word is a "neighbor"

1. Treat the target word *w* and a neighboring context word *c* as **positive examples**.
2. Randomly sample other words in the lexicon to get negative examples
3. Minimize $L(D, \overline{D}; \theta)$ to train *E*, *C* to distinguish those two cases
4. Use *E* for each word (often *E*+*C*) as its embedding

Quality of Representation Learning





Expression	*Nearest token*
Paris - France + Italy	Rome
bigger - big + cold	colder
sushi - Japan + Germany	bratwurst
Cu - copper + gold	Au
Windows - Microsoft + Google	Android
Montreal Canadiens - Montreal + Toronto	Toronto Maple Leafs

Solution to Fix N-Gram Language Models

- Increase N without increasing sparsity!
- Ideal: make N=all preceding words

- How? Deep Learning

☑ Goal #1: create a representation with low sparsity

- Goal #2: build an architecture that can use this information.

Neural Language Models

How to Build a Neural Language Model?

- Recall the Language Modeling task:
  - **Input:** sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$
  - **Output:** probability distribution of the next word $\boldsymbol{P}\left(\boldsymbol{x}^{(t+1)} \middle| \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}\right)$

- How about a window-based neural model?
- Important considerations
  - Instead of finding a good "middle" word, we will predict the "next" word
  - Instead of two class classification (D={0, 1}), we will model full prob. distribution over vocabulary

A Fixed-window Neural Language Model

output distribution

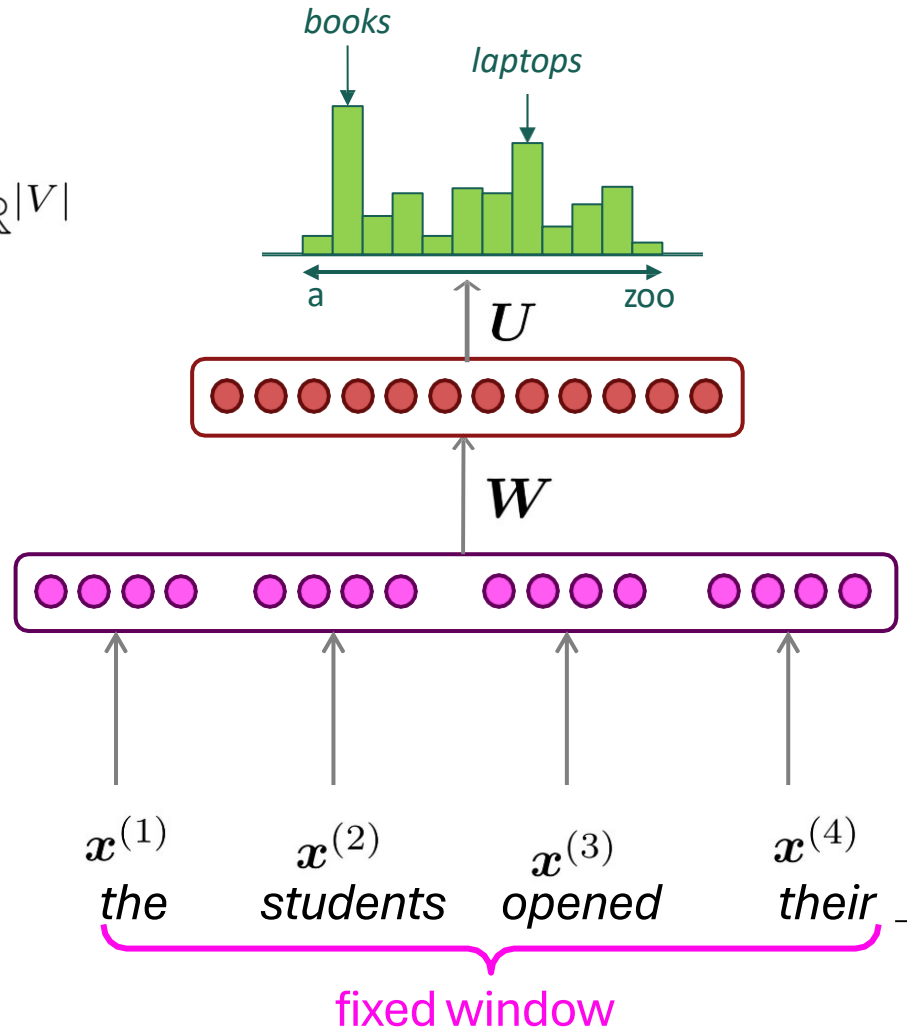
$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

$$\boldsymbol{h} = \sigma\left(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1\right)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$



~~as the proctor started the clock~~

discard

A Fixed-window Neural Language Model

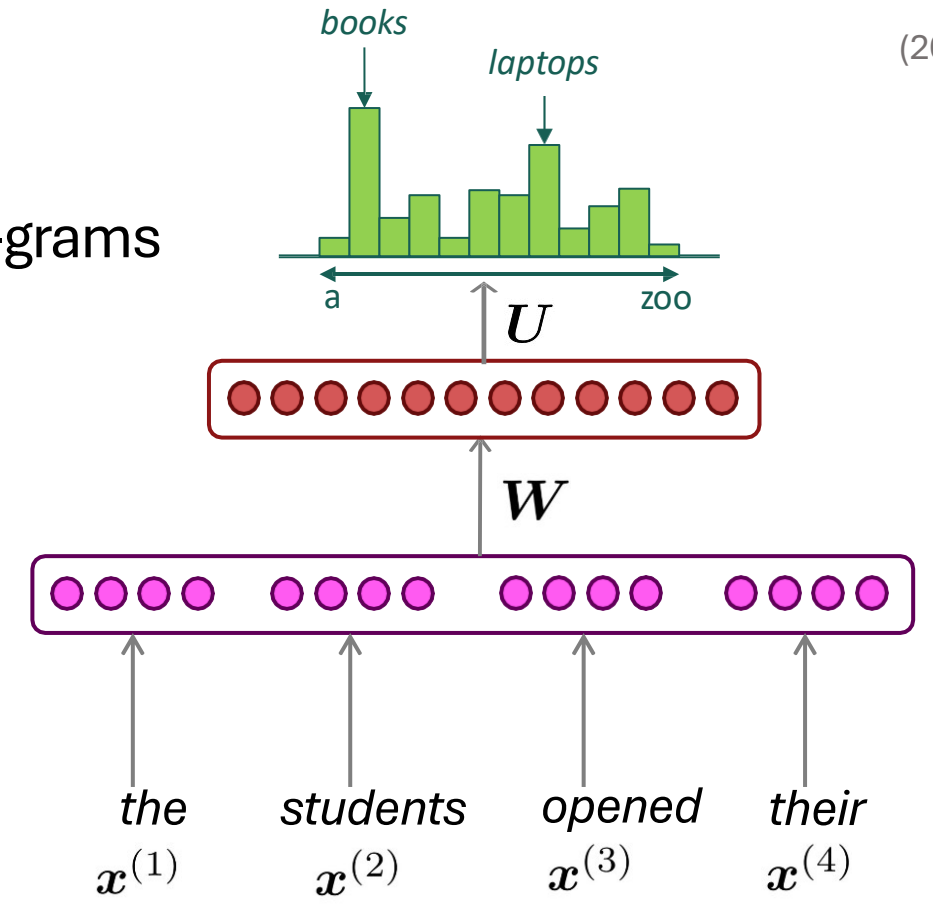
Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

Improvements over *n*-gram LM:

- No sparsity problem
- Don't need to store all observed *n*-grams

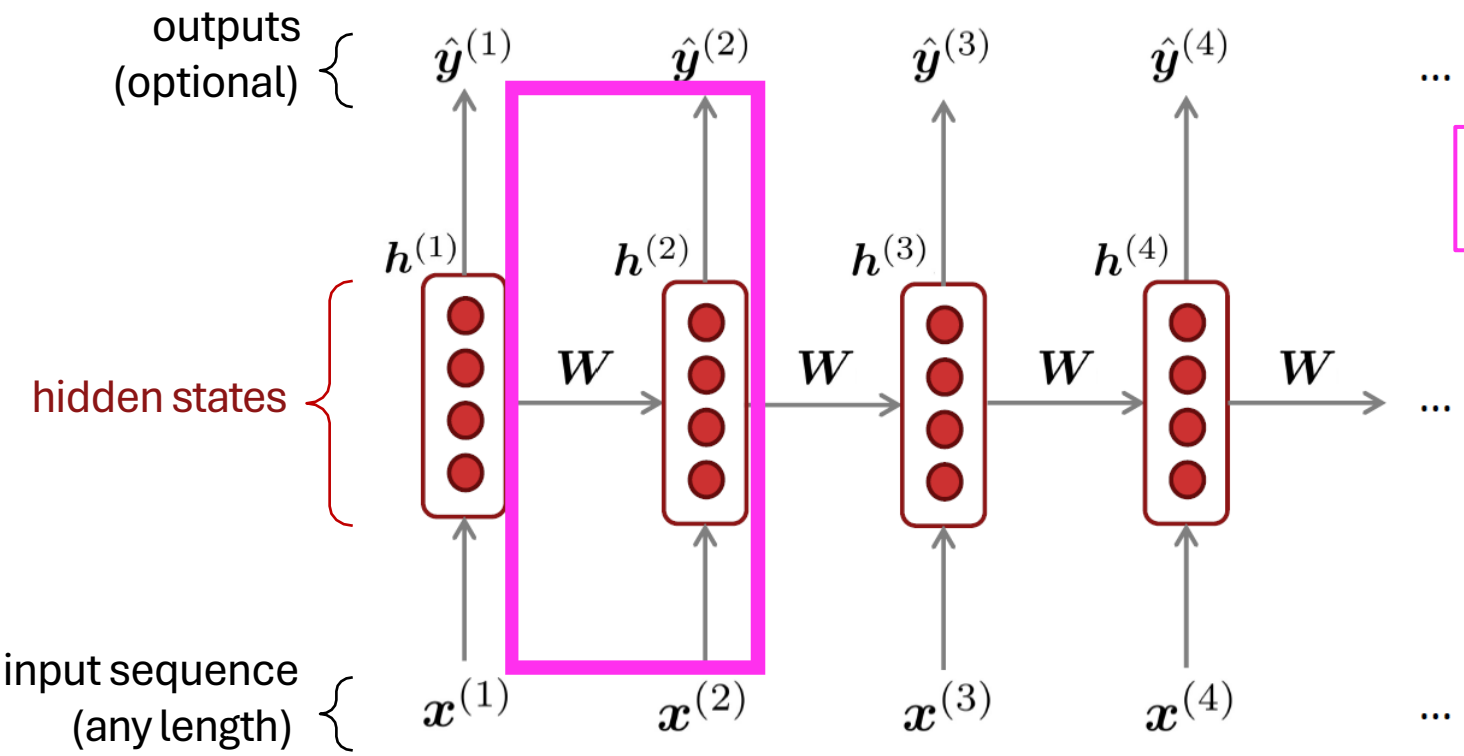
Remaining problems:

- Fixed window is too small
- Enlarging window enlarges W



We need a neural architecture that can process ***any length input***

Recurrent Neural Networks (RNN)



Core idea: Apply the same weights *W repeatedly*

A Simple RNN Language Model

output distribution

$$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}\left(\boldsymbol{U}\boldsymbol{h}^{(t)} + \boldsymbol{b}_2\right) \in \mathbb{R}^{|V|}$$

hidden states

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_e\boldsymbol{e}^{(t)} + \boldsymbol{b}_1\right)$$

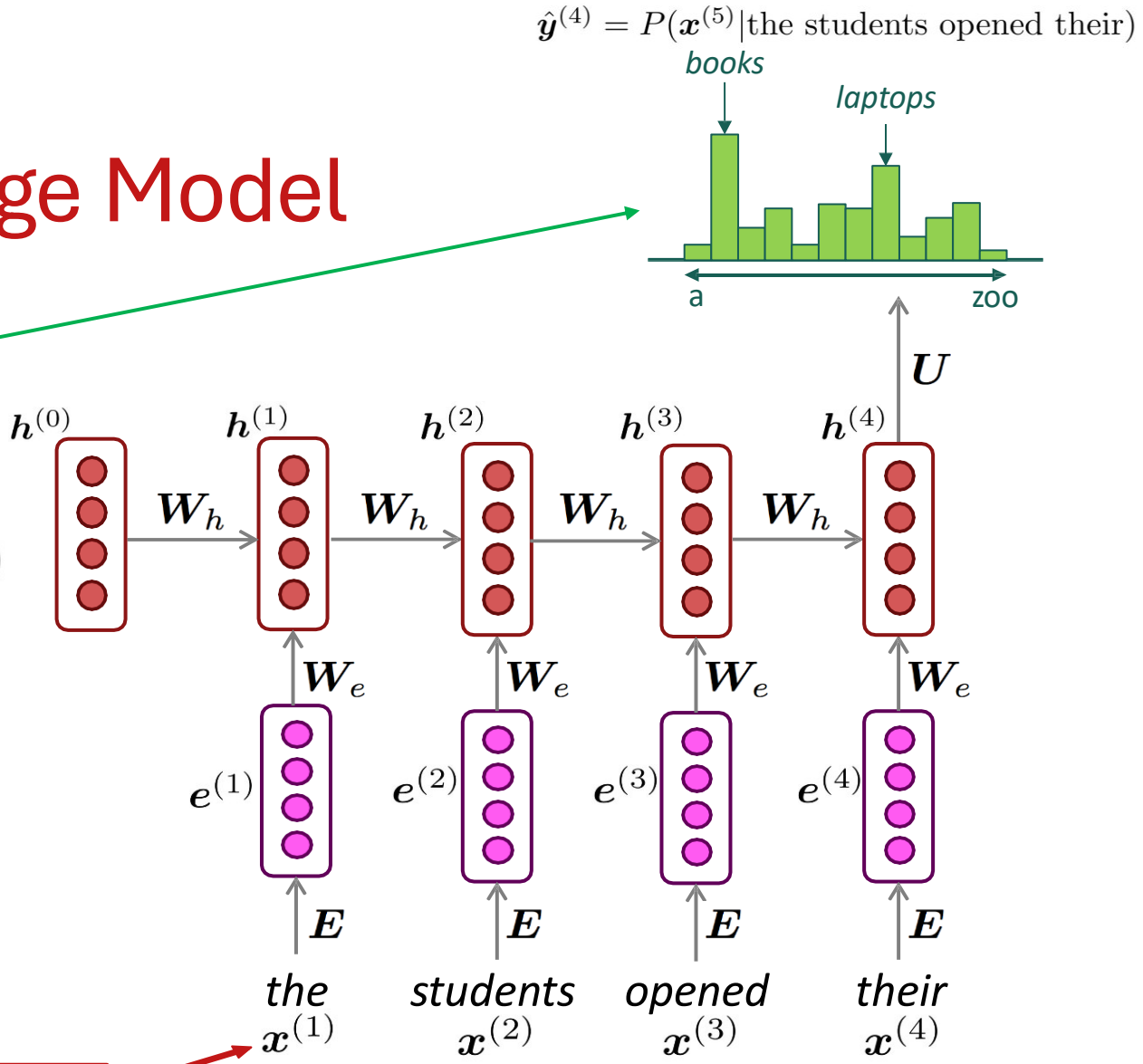
$\boldsymbol{h}^{(0)}$ is the initial hidden state

word embeddings

$$\boldsymbol{e}^{(t)} = \boldsymbol{E}\boldsymbol{x}^{(t)}$$

words / one-hot vectors

$$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$$



This input sequence could be much longer now!

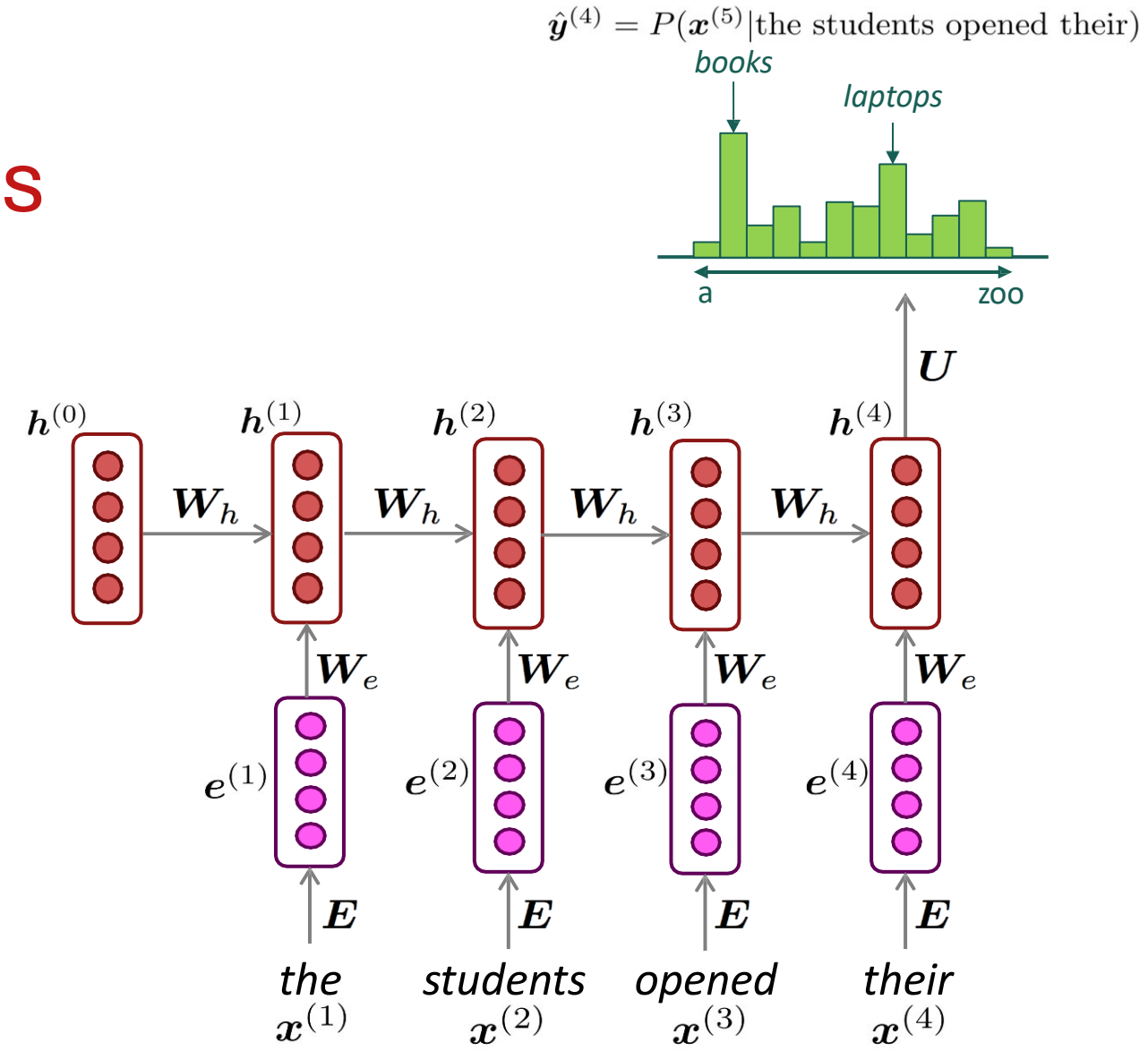
RNN Language Models

RNN Advantages:

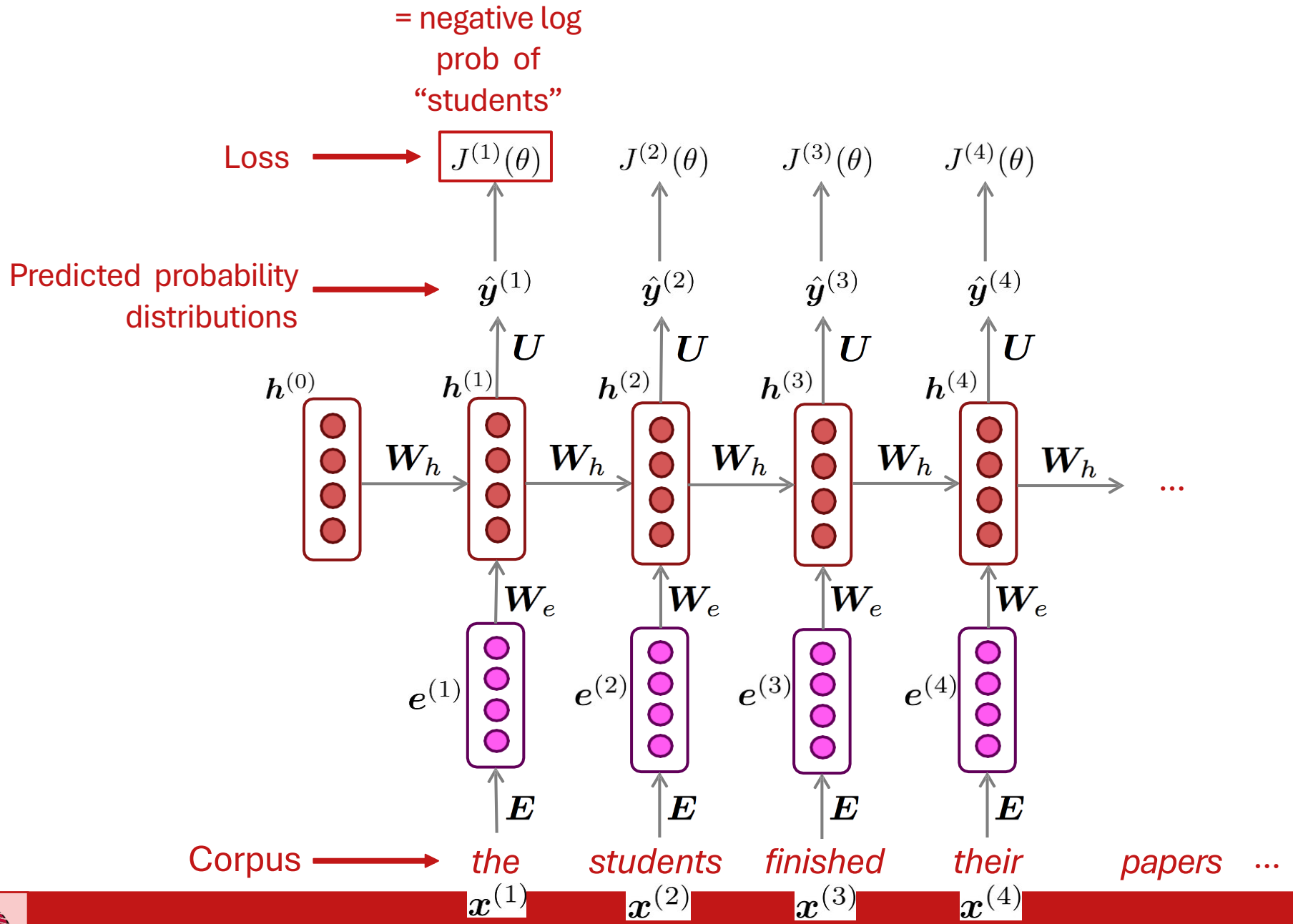
- Can process any length input
- Computation for step *t* can (in theory) use information from many steps back
- Model size doesn't increase for longer input context
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

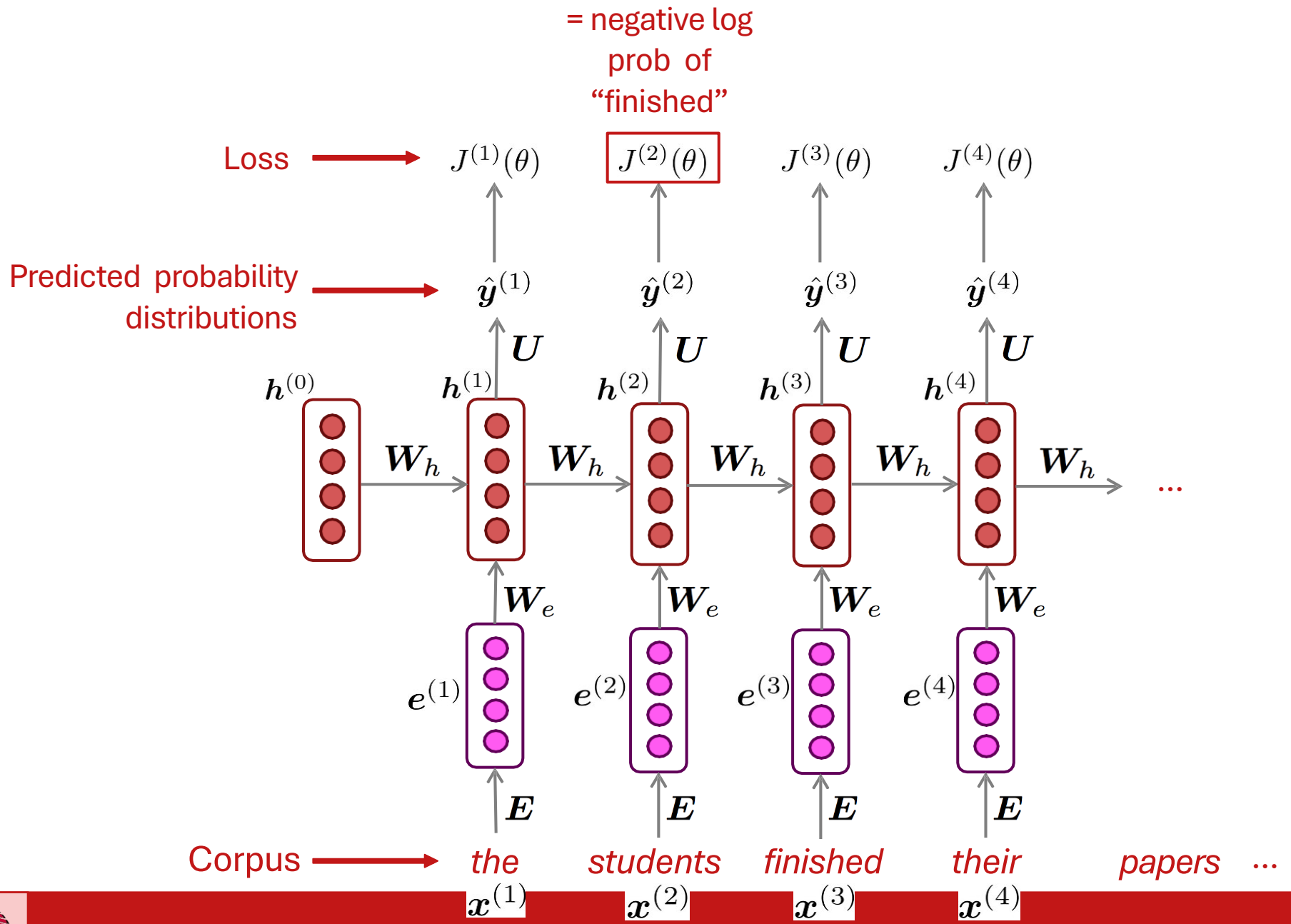
RNN Disadvantages:

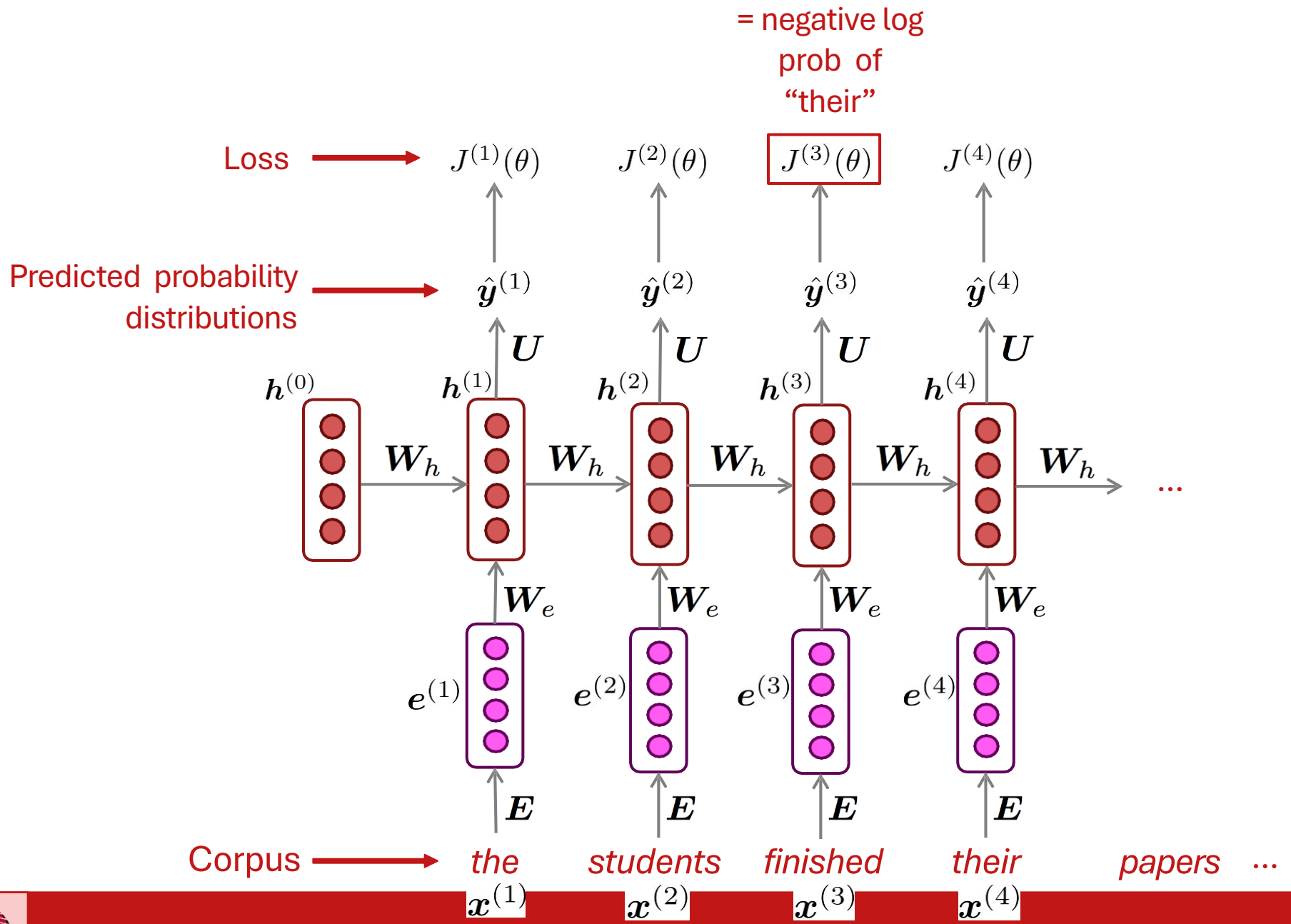
- Recurrent computation is slow
- In practice, difficult to access information from many steps back

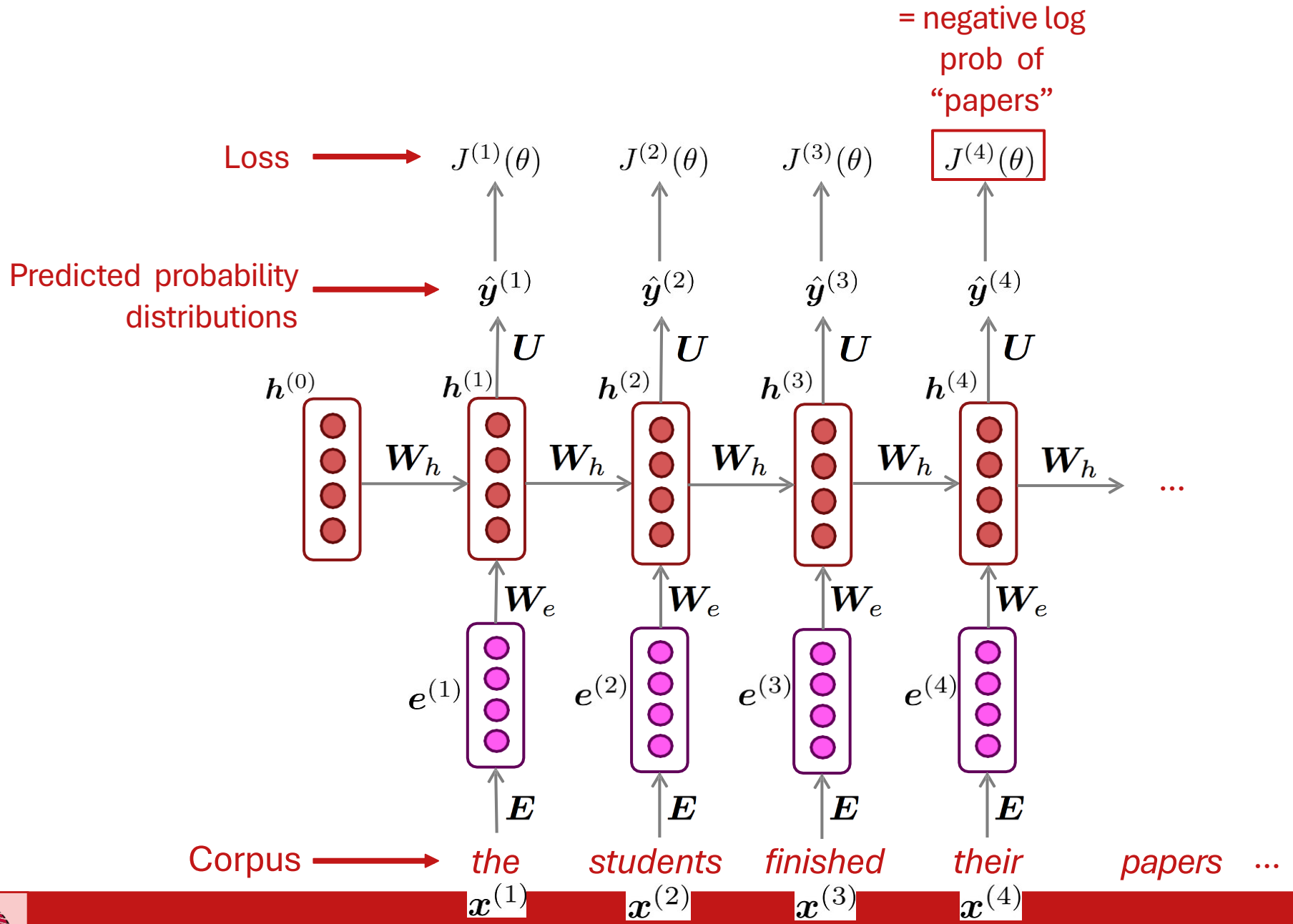


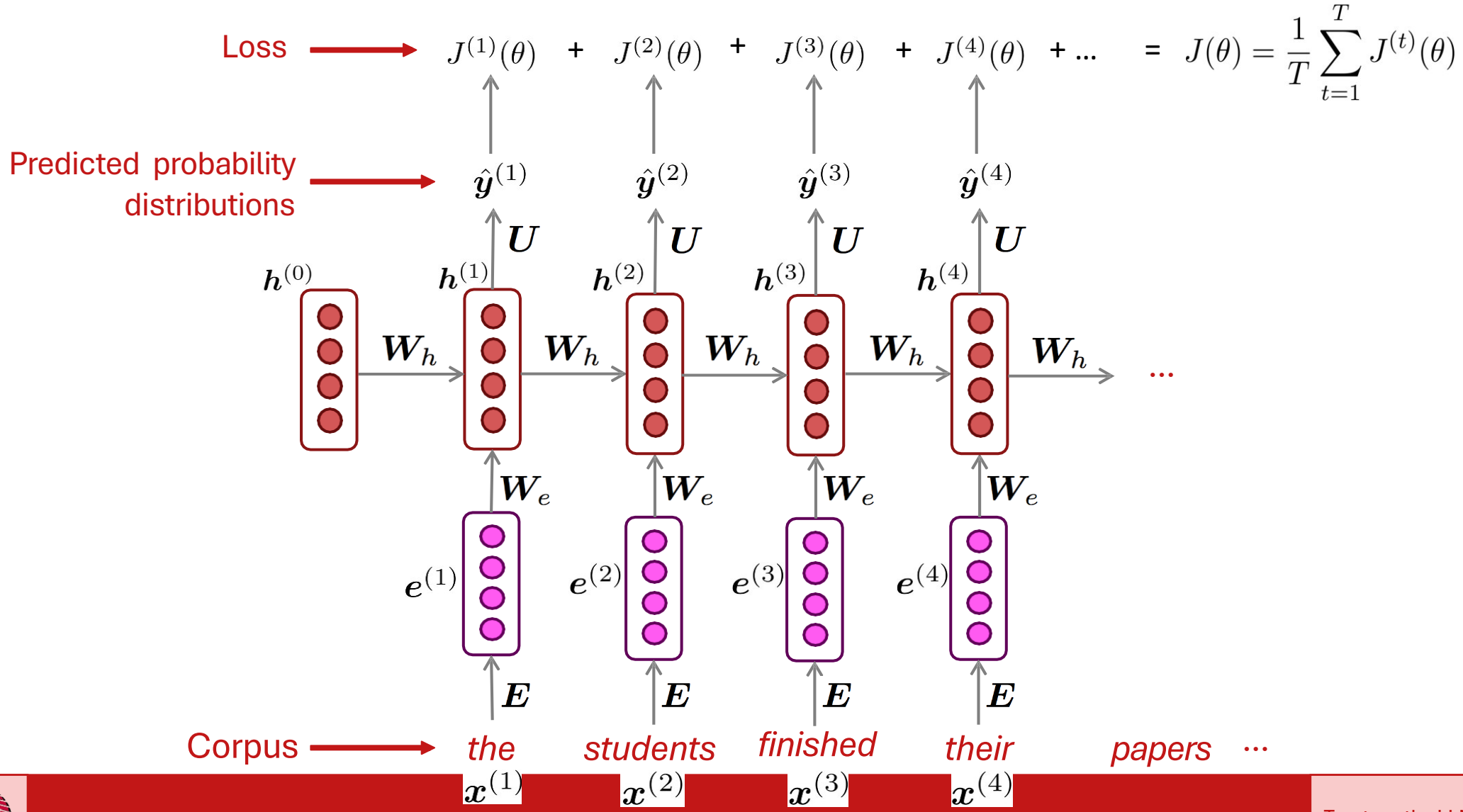
Training an RNN Language Model


= negative log prob of "students"
Loss
Predicted probability distributions
Corpus
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
$\hat{y}^{(1)}$
$\hat{y}^{(2)}$
$\hat{y}^{(3)}$
$\hat{y}^{(4)}$
U
$h^{(0)}$
$h^{(1)}$
$h^{(2)}$
$h^{(3)}$
$h^{(4)}$
W_h
...
W_e
$e^{(1)}$
$e^{(2)}$
$e^{(3)}$
$e^{(4)}$
E
the
students
finished
their
papers
...
$x^{(1)}$
$x^{(2)}$
$x^{(3)}$
$x^{(4)}$


= negative log prob of "finished"
Loss
Predicted probability distributions
Corpus
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
$\hat{y}^{(1)}$
$\hat{y}^{(2)}$
$\hat{y}^{(3)}$
$\hat{y}^{(4)}$
$h^{(0)}$
$h^{(1)}$
$h^{(2)}$
$h^{(3)}$
$h^{(4)}$
U
W_h
W_e
E
$e^{(1)}$
$e^{(2)}$
$e^{(3)}$
$e^{(4)}$
the
students
finished
their
papers
...
$x^{(1)}$
$x^{(2)}$
$x^{(3)}$
$x^{(4)}$

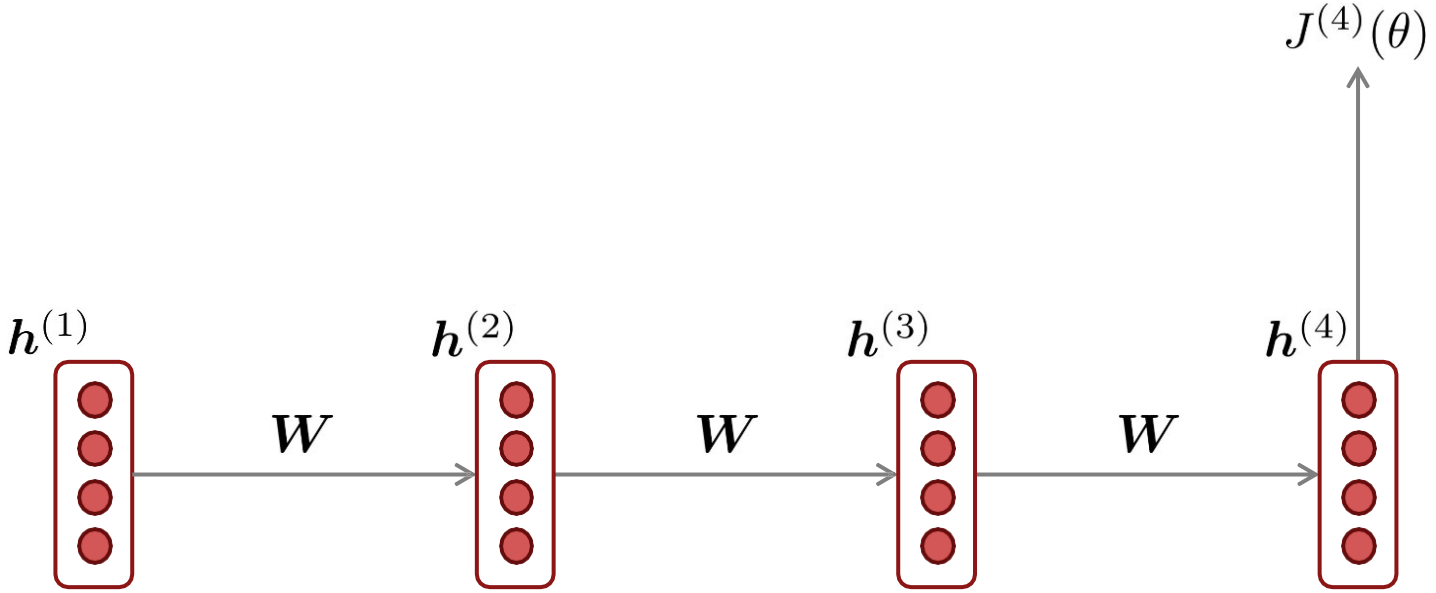

= negative log prob of "their"
Loss
Predicted probability distributions
Corpus
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
$\hat{y}^{(1)}$
$\hat{y}^{(2)}$
$\hat{y}^{(3)}$
$\hat{y}^{(4)}$
$h^{(0)}$
$h^{(1)}$
$h^{(2)}$
$h^{(3)}$
$h^{(4)}$
U
W_h
W_e
E
$e^{(1)}$
$e^{(2)}$
$e^{(3)}$
$e^{(4)}$
the
students
finished
their
papers
...
$x^{(1)}$
$x^{(2)}$
$x^{(3)}$
$x^{(4)}$


= negative log prob of "papers"
Loss
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
Predicted probability distributions
$\hat{y}^{(1)}$
$\hat{y}^{(2)}$
$\hat{y}^{(3)}$
$\hat{y}^{(4)}$
U
$h^{(0)}$
$h^{(1)}$
$h^{(2)}$
$h^{(3)}$
$h^{(4)}$
W_h
...
W_e
$e^{(1)}$
$e^{(2)}$
$e^{(3)}$
$e^{(4)}$
E
Corpus
the
students
finished
their
papers
...
$x^{(1)}$
$x^{(2)}$
$x^{(3)}$
$x^{(4)}$

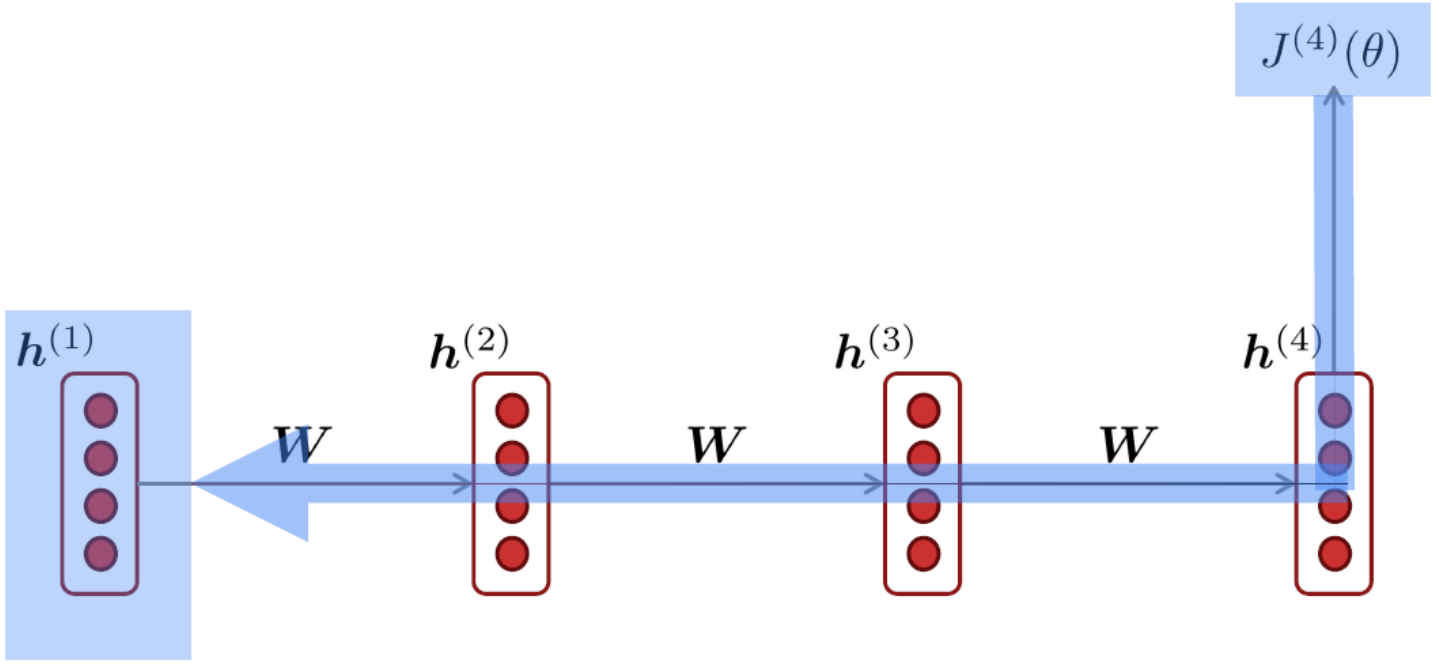

Loss
$J^{(1)}(\theta)$ + $J^{(2)}(\theta)$ + $J^{(3)}(\theta)$ + $J^{(4)}(\theta)$ + ... = $J(\theta) = \frac{1}{T}\sum_{t=1}^{T} J^{(t)}(\theta)$
Predicted probability distributions
$\hat{y}^{(1)}$ $\hat{y}^{(2)}$ $\hat{y}^{(3)}$ $\hat{y}^{(4)}$
U
$h^{(0)}$ $h^{(1)}$ $h^{(2)}$ $h^{(3)}$ $h^{(4)}$
W_h
...
W_e
$e^{(1)}$ $e^{(2)}$ $e^{(3)}$ $e^{(4)}$
E
Corpus
the students finished their papers ...
$x^{(1)}$ $x^{(2)}$ $x^{(3)}$ $x^{(4)}$

Problems with RNNs

Vanishing and Exploding Gradients

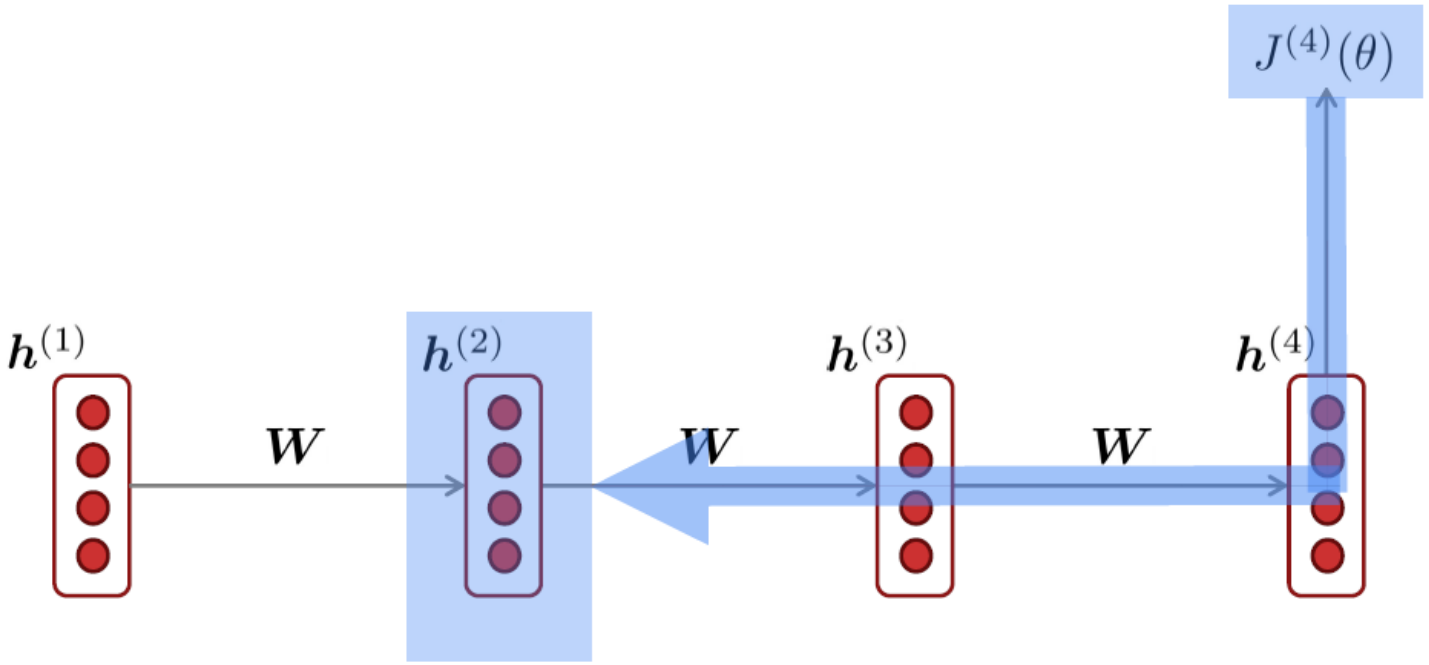


Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \ ?$$

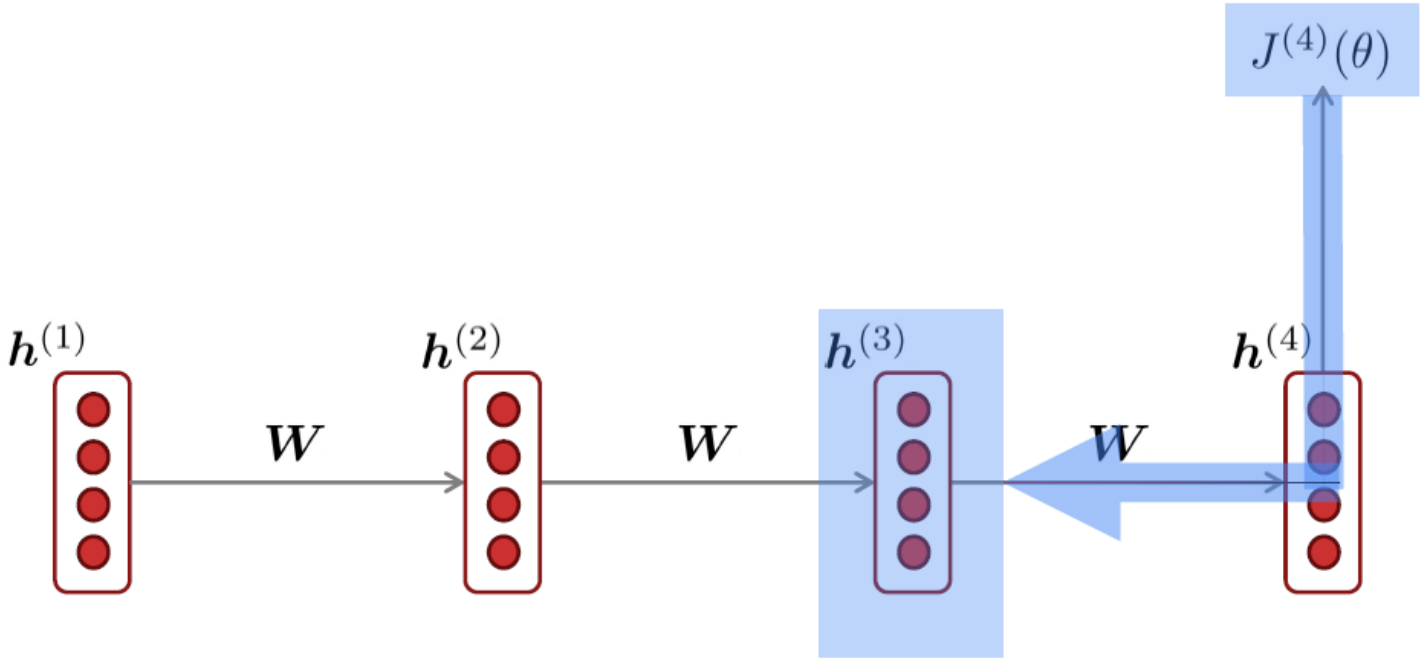
Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial J^{(4)}}{\partial h^{(2)}}$$

Chain Rule!

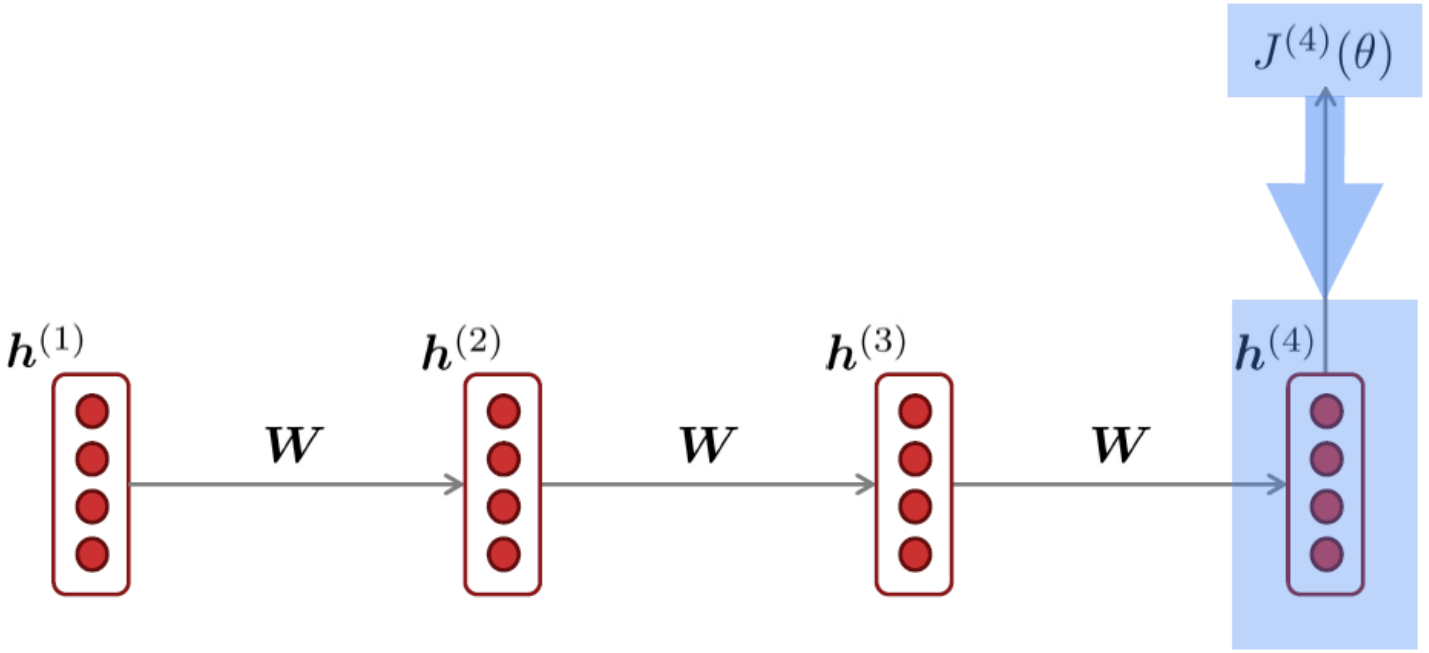
Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial J^{(4)}}{\partial h^{(3)}}$$

Chain Rule!

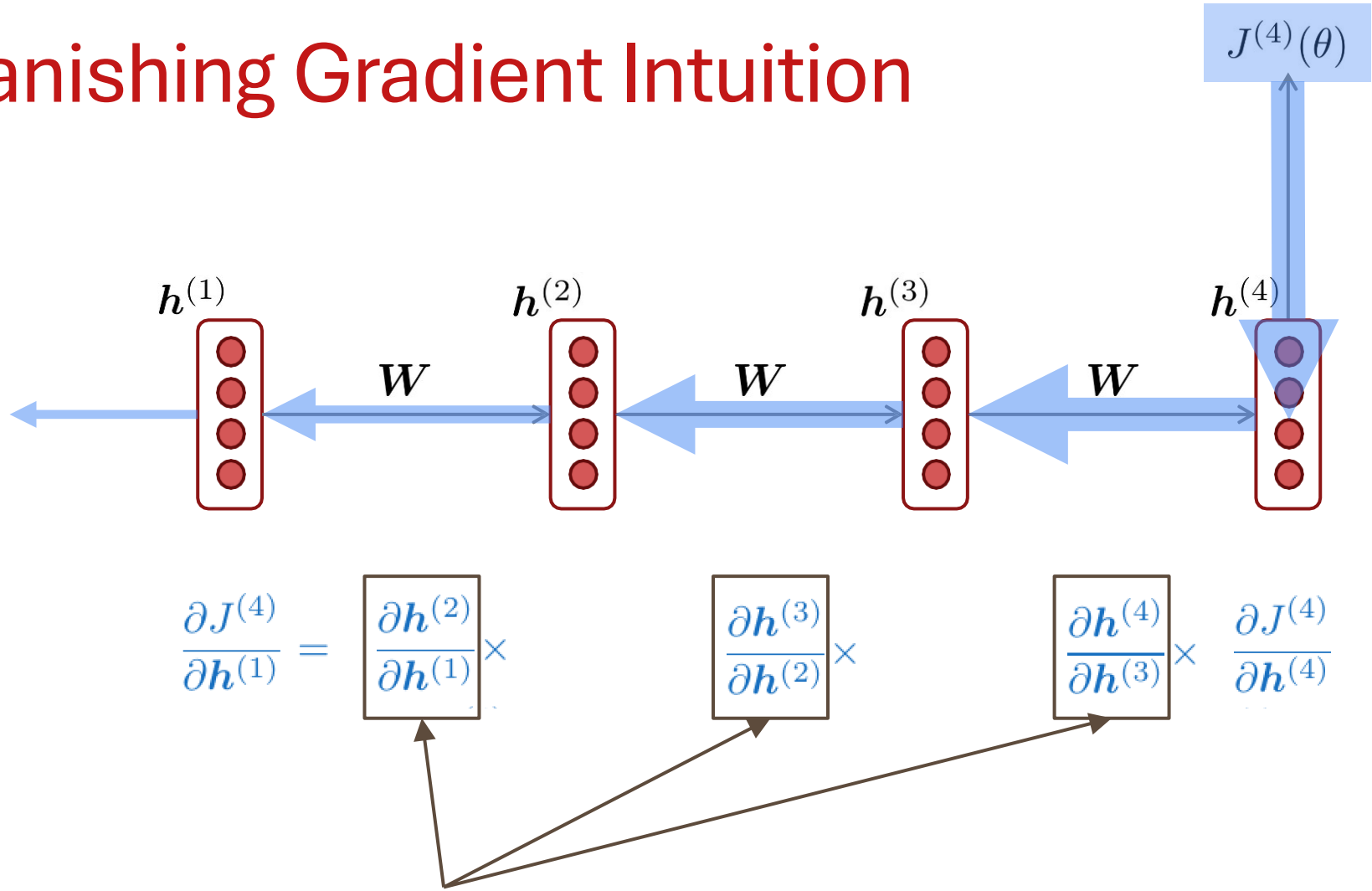
Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

Chain Rule!

Vanishing Gradient Intuition



What happens if these are small?

Vanishing gradient problem: When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Effect of Vanishing Gradient on RNN-LM

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her* _____
- To learn from this training example, the RNN-LM needs to model the dependency between *"tickets"* on the 7th step and the target word *"tickets"* at the end.
- But if the gradient is small, the model can't learn this dependency
  - So, the model is unable to predict similar long-distance dependencies at test time

LSTMs & GRUs

Improving RNN-LM

output distribution

$$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}\left(\boldsymbol{U}\boldsymbol{h}^{(t)} + \boldsymbol{b}_2\right) \in \mathbb{R}^{|V|}$$

hidden states

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_e\boldsymbol{e}^{(t)} + \boldsymbol{b}_1\right)$$

$\boldsymbol{h}^{(0)}$ is the initial hidden state

- Think of simple RNN as a computer. Input ($e^{(t)}$) arrives. Memory h gets updated
- In simple RNNs entire memory is rewritten at every time step!
  - There is no explicit inertia!
- Memory predicts the output PLUS maintains the history
  - Ideally those two calculations should be separated.

Selectivity to Control Writing

- Write Selectively: when taking class notes, we only record the most important points from the input
- Read Selectively: focus on most relevant knowledge for the output
- Forget Selectively: in order to make room for new information, we need to selectively forget the least relevant old information

LSTM

We have a sequence of inputs $x^{(t)}$, and we will compute a sequence of hidden states $h^{(t)}$ and cell states $c^{(t)}$. On timestep *t*:

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the proposed cell state to write to cell

Output gate: controls what parts of cell are output to hidden state

Sigmoid function: all gate values are between 0 and 1

$$\boldsymbol{f}^{(t)} = \sigma\left(\boldsymbol{W}_f \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_f \boldsymbol{x}^{(t)} + \boldsymbol{b}_f\right)$$

$$\boldsymbol{i}^{(t)} = \sigma\left(\boldsymbol{W}_i \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_i \boldsymbol{x}^{(t)} + \boldsymbol{b}_i\right)$$

$$\boldsymbol{o}^{(t)} = \sigma\left(\boldsymbol{W}_o \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_o \boldsymbol{x}^{(t)} + \boldsymbol{b}_o\right)$$

New cell content: this is the new content to be written to the cell

Cell state: erase ("forget") some content from last cell state, and write ("input") some new cell content

Hidden state: read ("output") some content from the cell

$$\tilde{\boldsymbol{c}}^{(t)} = \tanh\left(\boldsymbol{W}_c \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_c \boldsymbol{x}^{(t)} + \boldsymbol{b}_c\right)$$

$$\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \odot \boldsymbol{c}^{(t-1)} + \boldsymbol{i}^{(t)} \odot \tilde{\boldsymbol{c}}^{(t)}$$

$$\boldsymbol{h}^{(t)} = \boldsymbol{o}^{(t)} \odot \tanh \boldsymbol{c}^{(t)}$$

All these are vectors of same length *n*

Gates are applied using element-wise (or Hadamard) product: $\odot$

Less Problem of Vanishing Gradient

$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)}$$

$$f_t = \sigma\left(W_f h^{(t-1)} + U_f x^{(t)} + b_f\right)$$

$$\frac{\partial c^{(t)}}{\partial c^{(t-1)}} = \frac{\partial f^{(t)}}{\partial c^{(t-1)}} c^{(t-1)} + \frac{\partial c^{(t-1)}}{\partial c^{(t-1)}} f^{(t)} + \frac{\partial i^{(t)}}{\partial c^{(t-1)}} \tilde{c}^{(t)} + \frac{\partial \tilde{c}^{(t)}}{\partial c^{(t-1)}} i^{(t)}$$

Initialize such that $f^{(t)} \rightarrow 1$

$\rightarrow$ b_f = 1 or more

Gated Recurrent Units (GRUs)

- Proposed by Cho et al. in 2014 as a simpler alternative to the LSTM.
- On each timestep *t*, we have input $x^{(t)}$ and hidden state $h^{(t)}$ (no cell state).

How does this solve vanishing gradient? Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate bias to >1)

Update gate: controls what parts of hidden state are updated vs preserved

$$\boldsymbol{u}^{(t)} = \sigma\left(\boldsymbol{W}_u \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_u \boldsymbol{x}^{(t)} + \boldsymbol{b}_u\right)$$

Reset gate: controls what parts of previous hidden state are used to compute new content

$$\boldsymbol{r}^{(t)} = \sigma\left(\boldsymbol{W}_r \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_r \boldsymbol{x}^{(t)} + \boldsymbol{b}_r\right)$$

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

$$\tilde{\boldsymbol{h}}^{(t)} = \tanh\left(\boldsymbol{W}_h(\boldsymbol{r}^{(t)} \circ \boldsymbol{h}^{(t-1)}) + \boldsymbol{U}_h \boldsymbol{x}^{(t)} + \boldsymbol{b}_h\right)$$

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

$$\boldsymbol{h}^{(t)} = (1 - \boldsymbol{u}^{(t)}) \circ \boldsymbol{h}^{(t-1)} + \boldsymbol{u}^{(t)} \circ \tilde{\boldsymbol{h}}^{(t)}$$

"Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", Cho et al. 2014, https://arxiv.org/pdf/1406.1078v3.pdf

Solution to Fix N-Gram Language Models

- Increase N without increasing sparsity!

☑ Ideal: make N=all preceding words

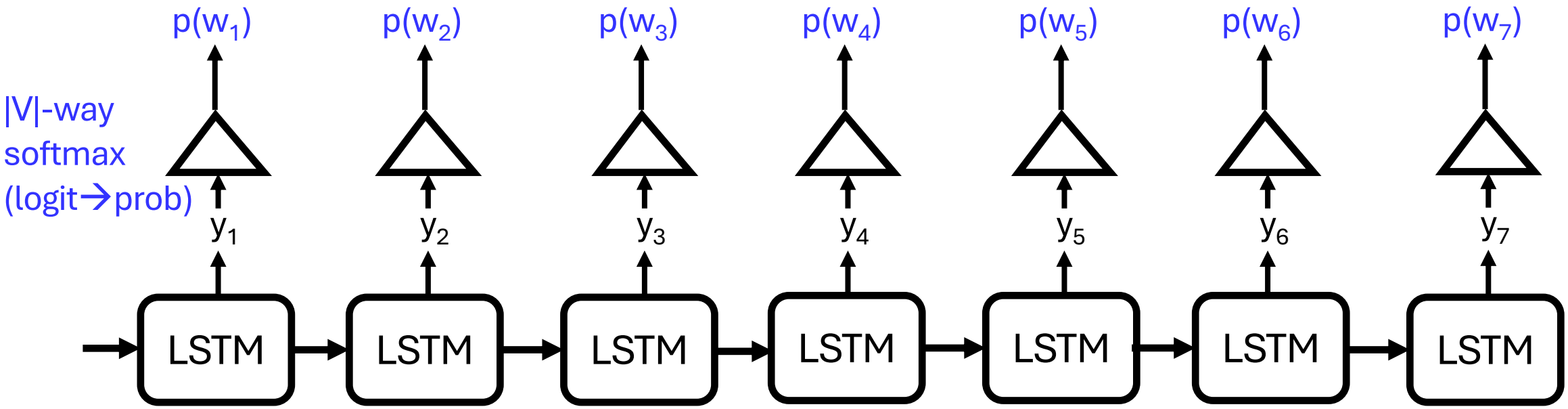
- How? Deep Learning

☑ Goal #1: create a representation with low sparsity

☑ Goal #2: build an architecture that can use this information.

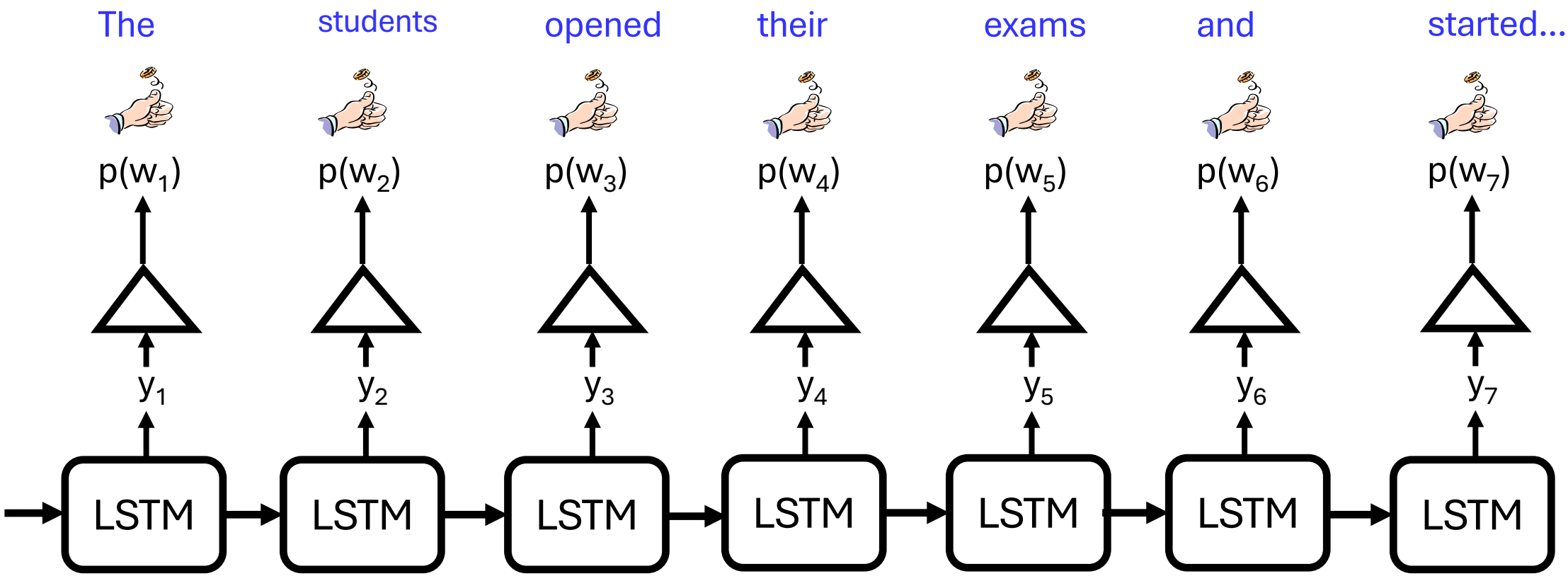
Generation from an LM

Neural Language Model



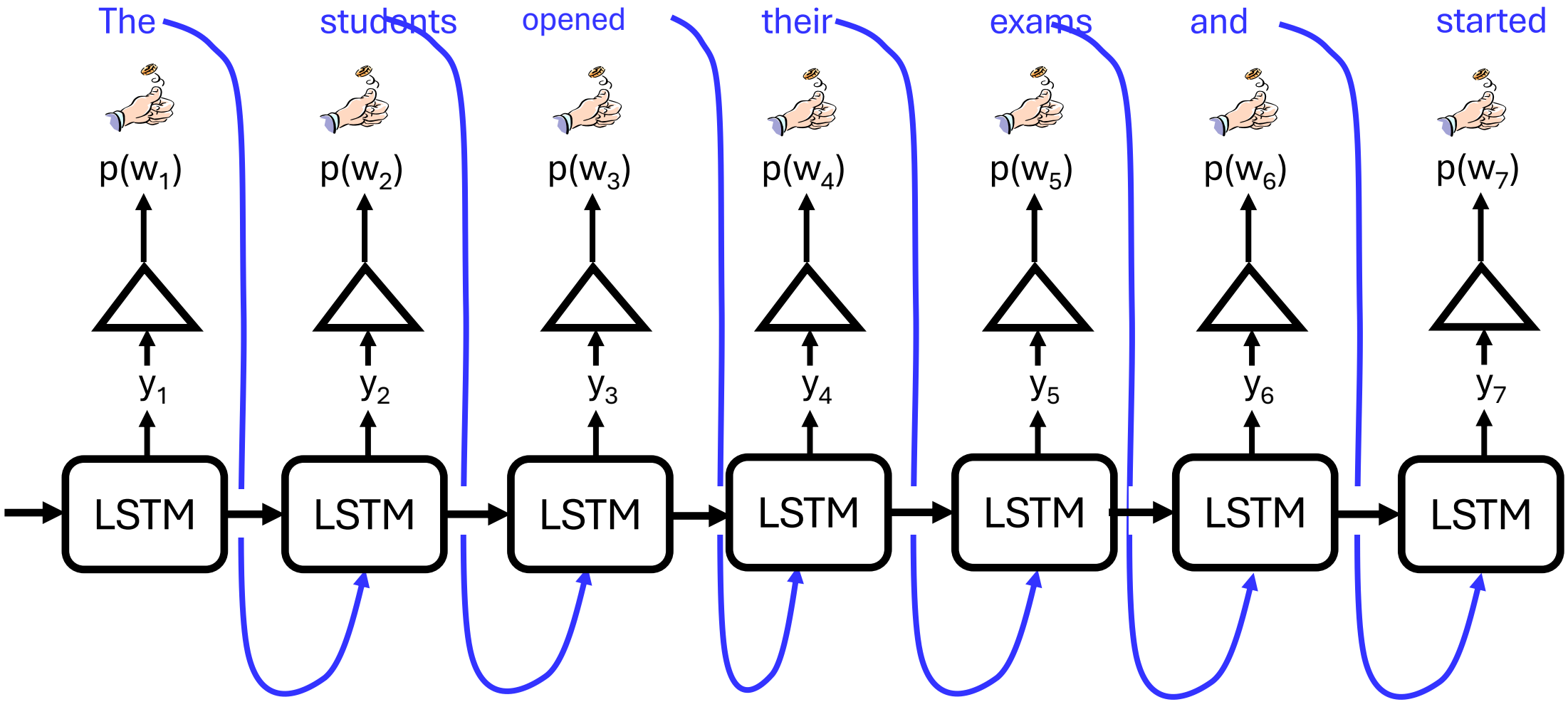
How do we get the actual sentence from a sequence of probability distributions?

Sampling from a Neural LM

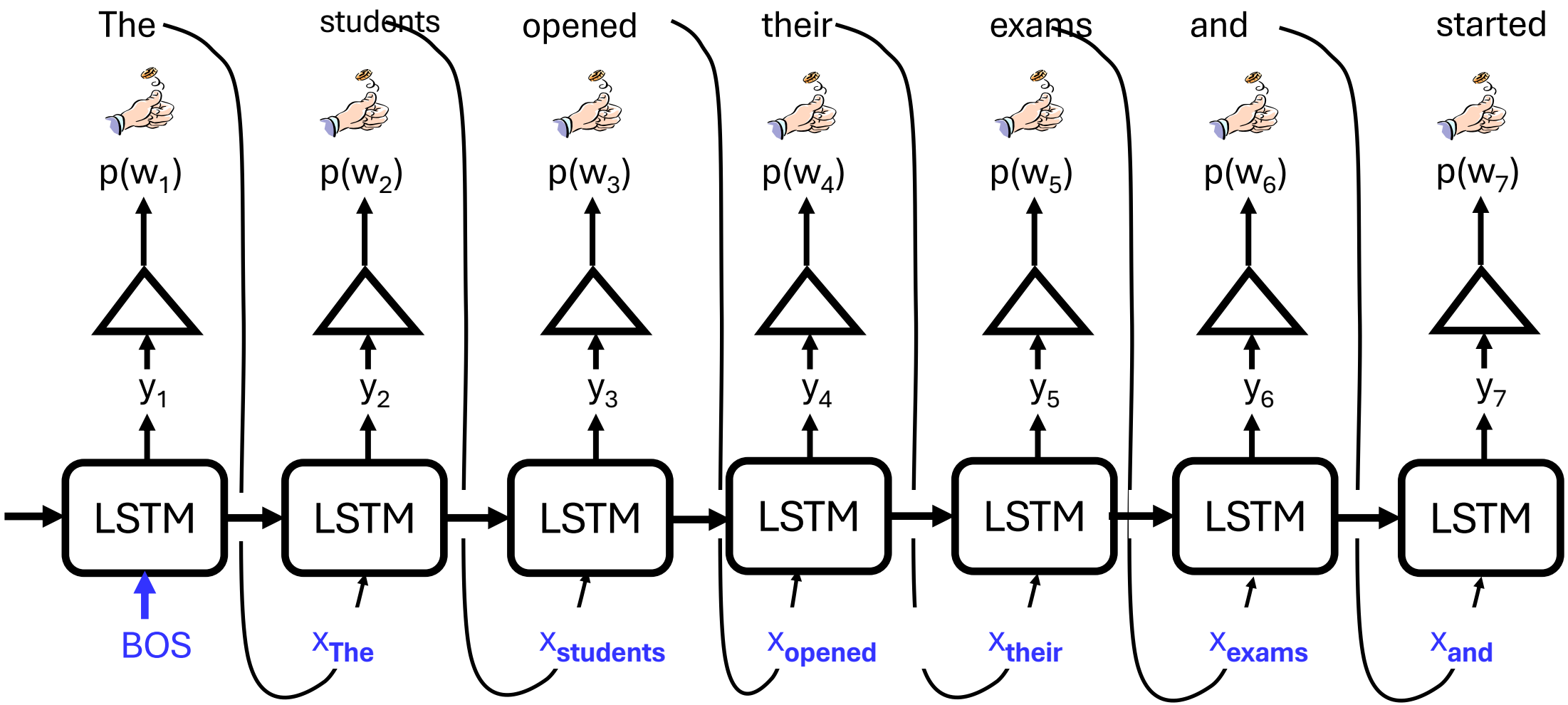


Can you think of a fundamental problem in this design?

Sampling from a Neural LM

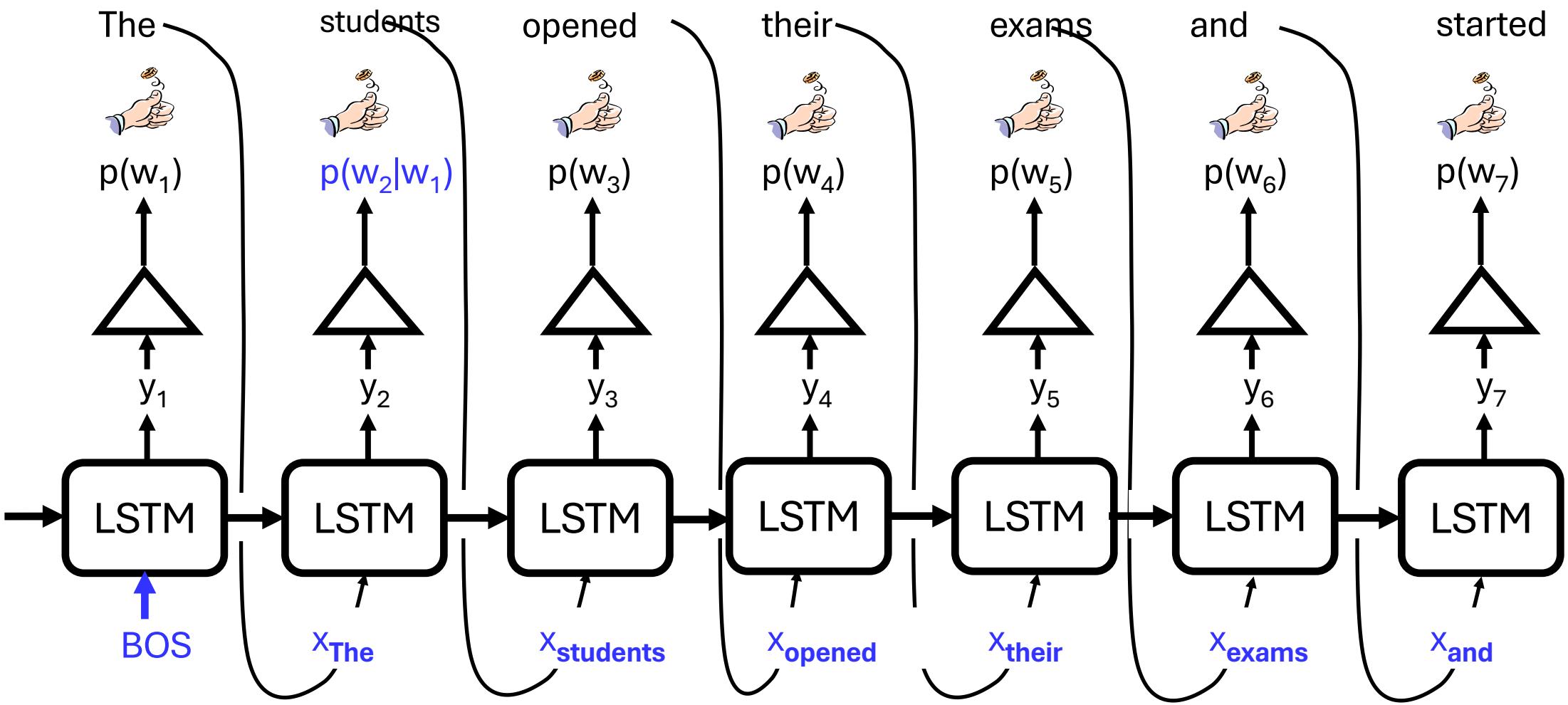


Sampling from a Neural LM



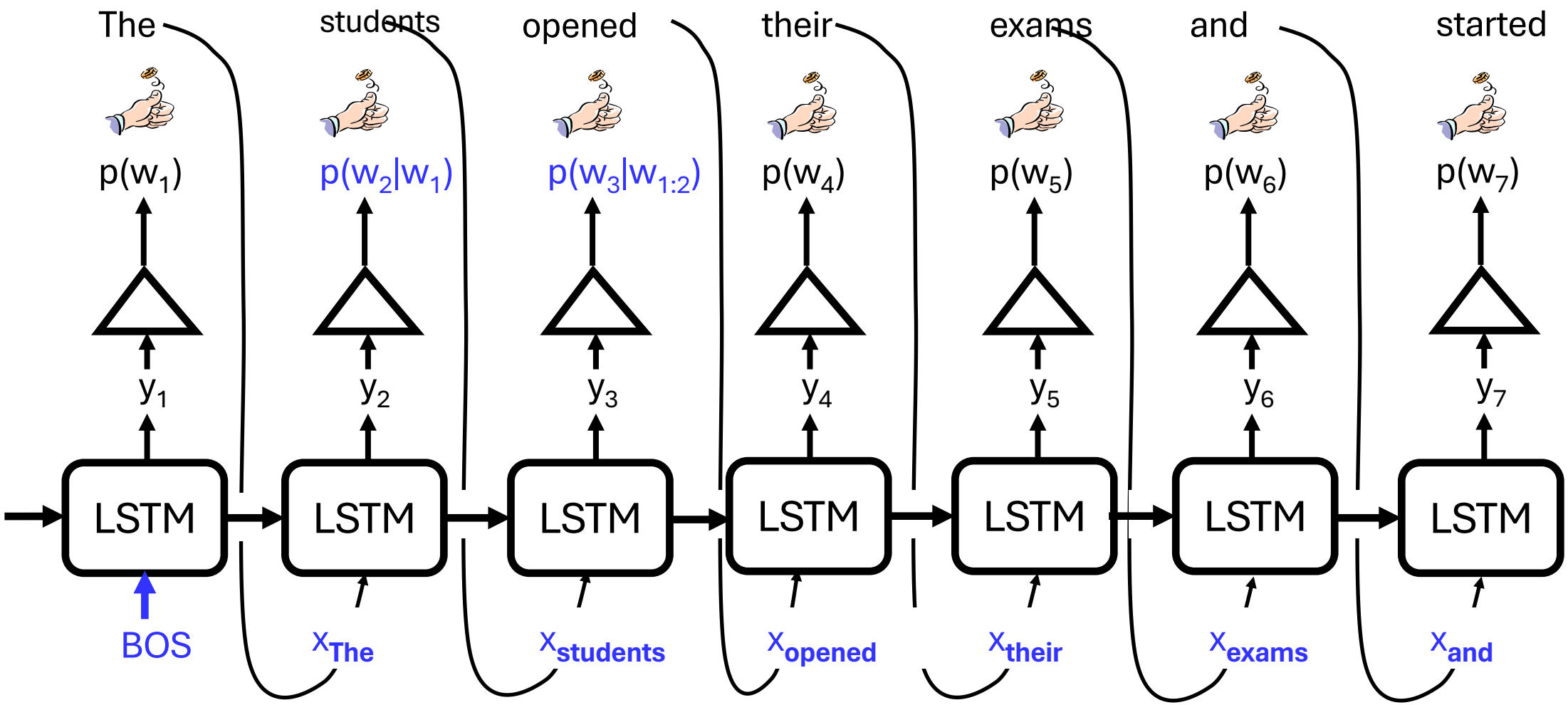
Called "Auto-regressive models"

Sampling from a Neural LM

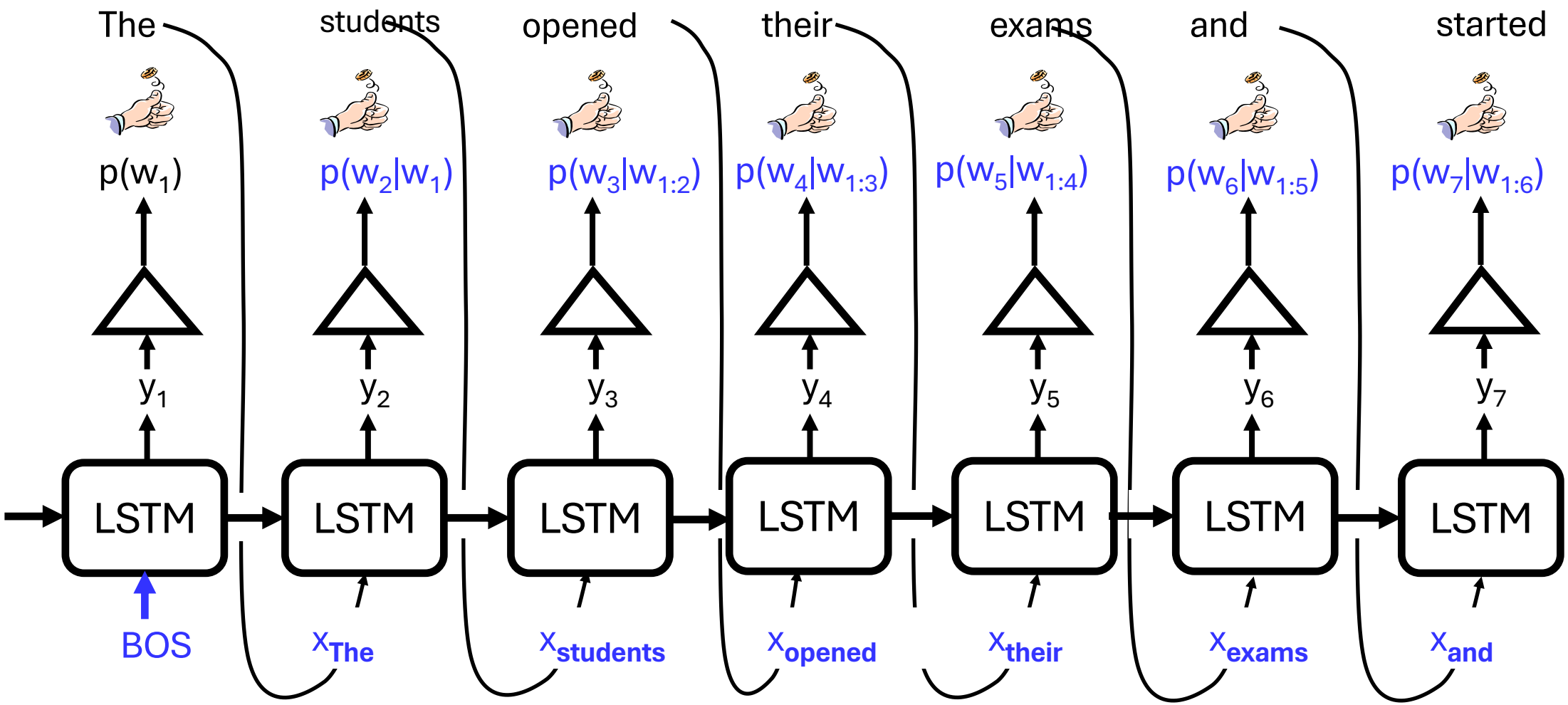


Called "Auto-regressive models"

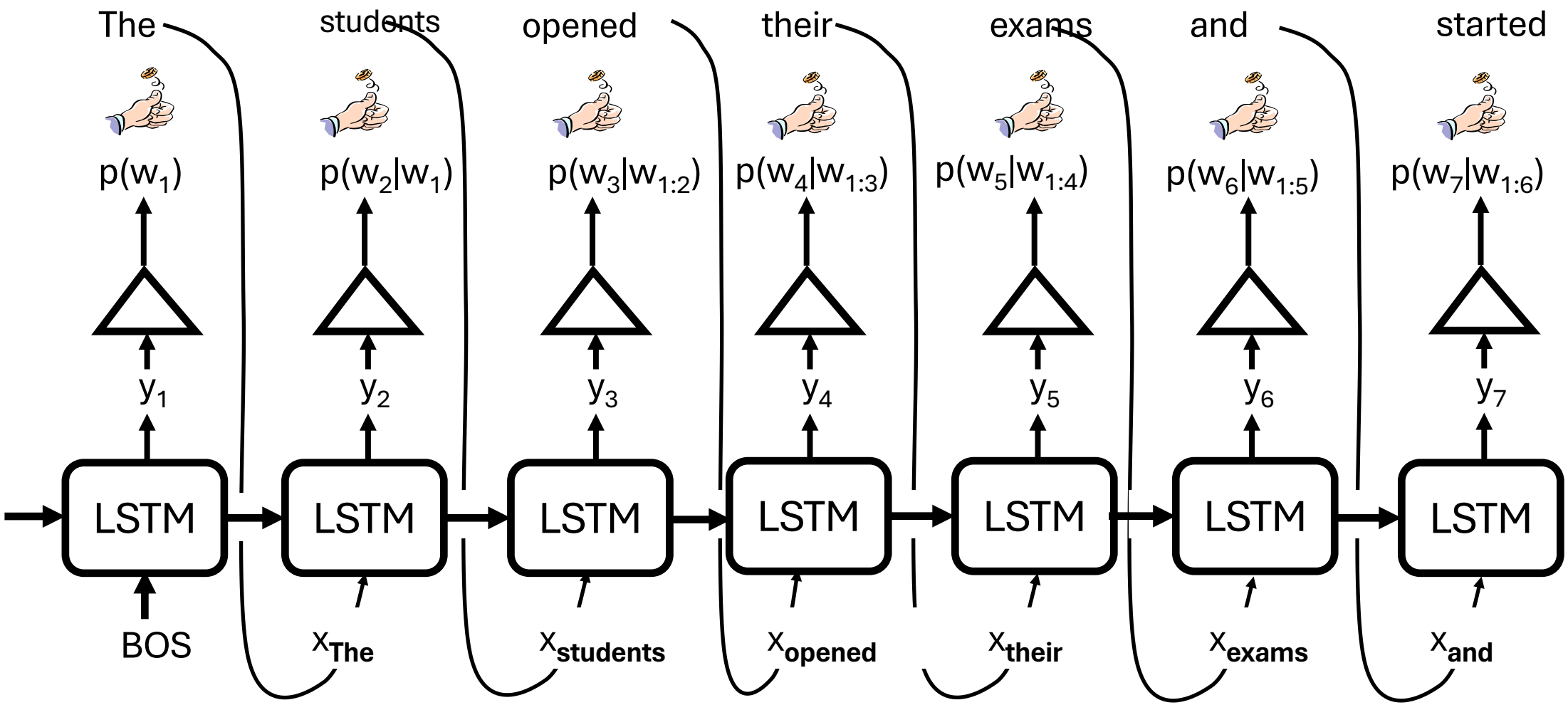
Sampling from a Neural LM



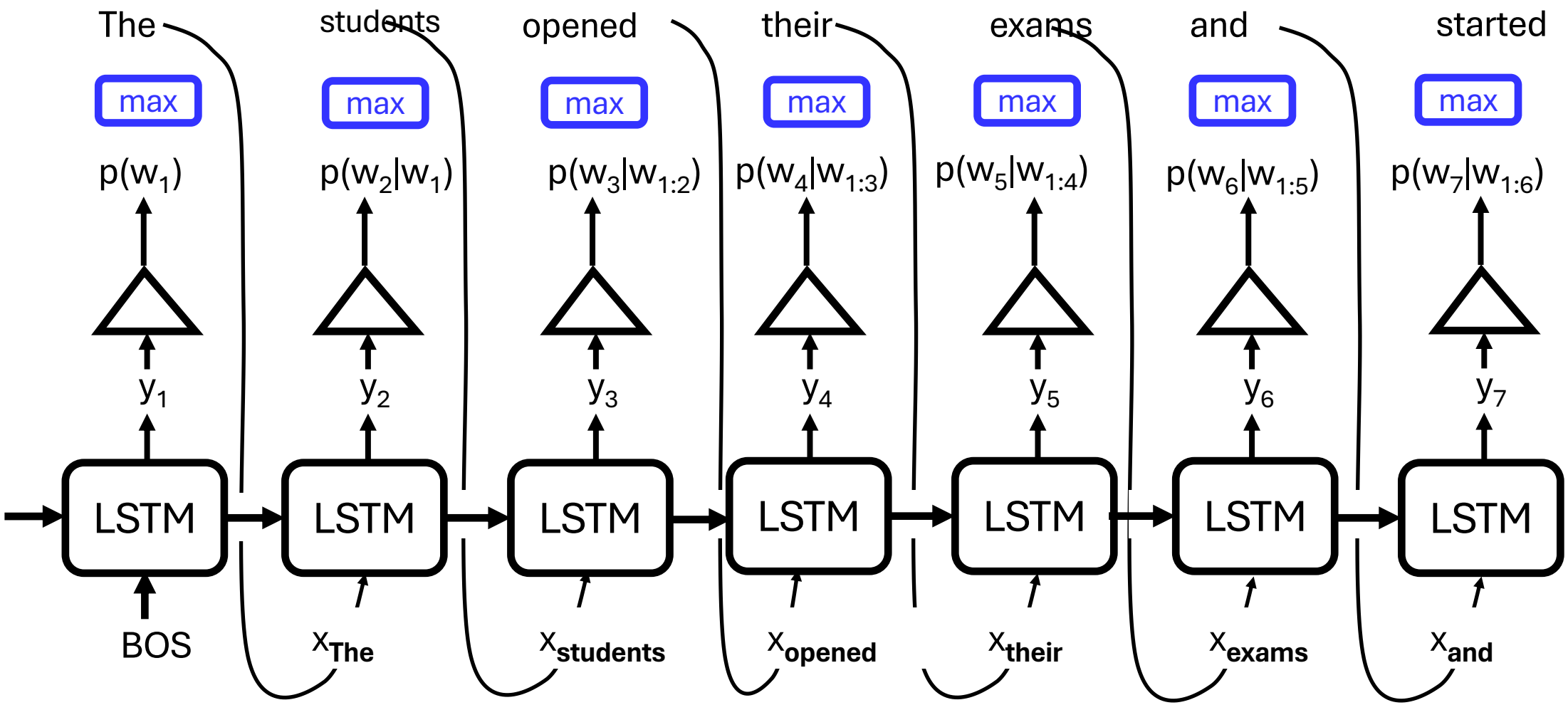
Sampling from a Neural LM (Autoregressive Models)



Sampling vs. Greedy



Sampling vs. Greedy



Sampling: Limitations

- Problem: makes every token in the vocabulary an option at every step
  - Even if most probability mass in the distribution is over a limited set of options
  - The tail of the distribution could be very long and in aggregate have considerable mass (statistics: “heavy tailed” distributions)

- Many tokens are probably really wrong in the current context.
  - Although each of them may be assigned a small probability, in aggregate they still get a high chance to be selected.

Greedy: Limitations

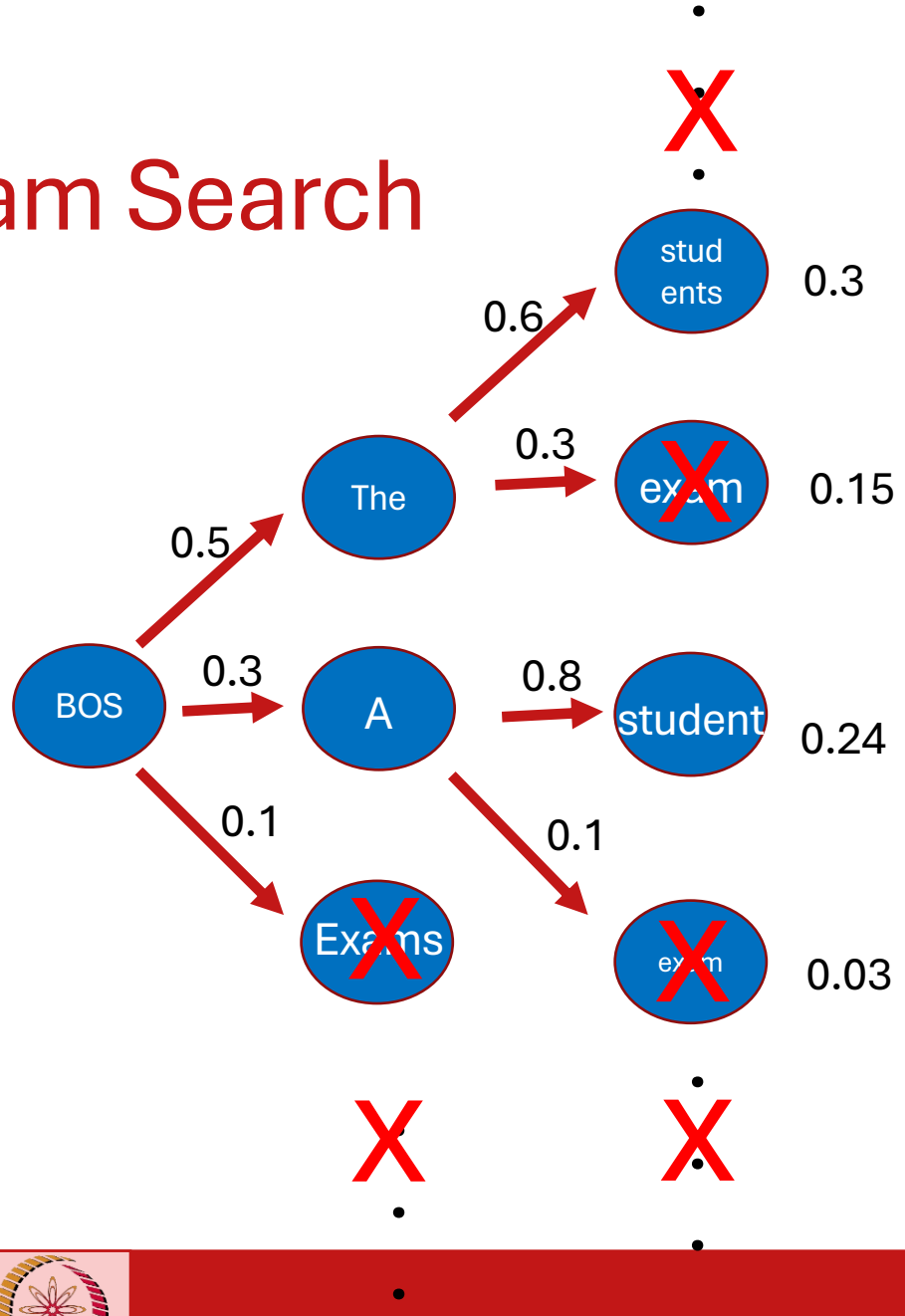
- Text has no diversity (takes max at every step)

- Text is still not the highest probability! Why?
- What is the function we actually wish to compute?

$$\underset{w_{1:n}}{\operatorname{argmax}} P(w_{1:n})$$

- Computing this expression is prohibitive.
- Greedy Approach: approximation can be bad because
  - model will never begin a sentence with a low probability word
  - model will prefer many common words to one rare word
- Solution: Beam Search

Beam Search



Instead of picking one greedy path, maintain multiple greedy paths

Upto a constant beam of b

Example for beam size =2

Beam Search Problems

- Expensive!
  - O(beam size x vocabulary size x generation length)
- Lack of Diversity

- Repetition in Tokens

Beam Search: Repetition in Tokens [Holtzman et al ICLR'20]

- Context: In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.
- Continuation: The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the **Universidad Nacional Autónoma de México (UNAM)** and the Universidad Nacional Autónoma de México (UNAM/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México…

Simple Solutions

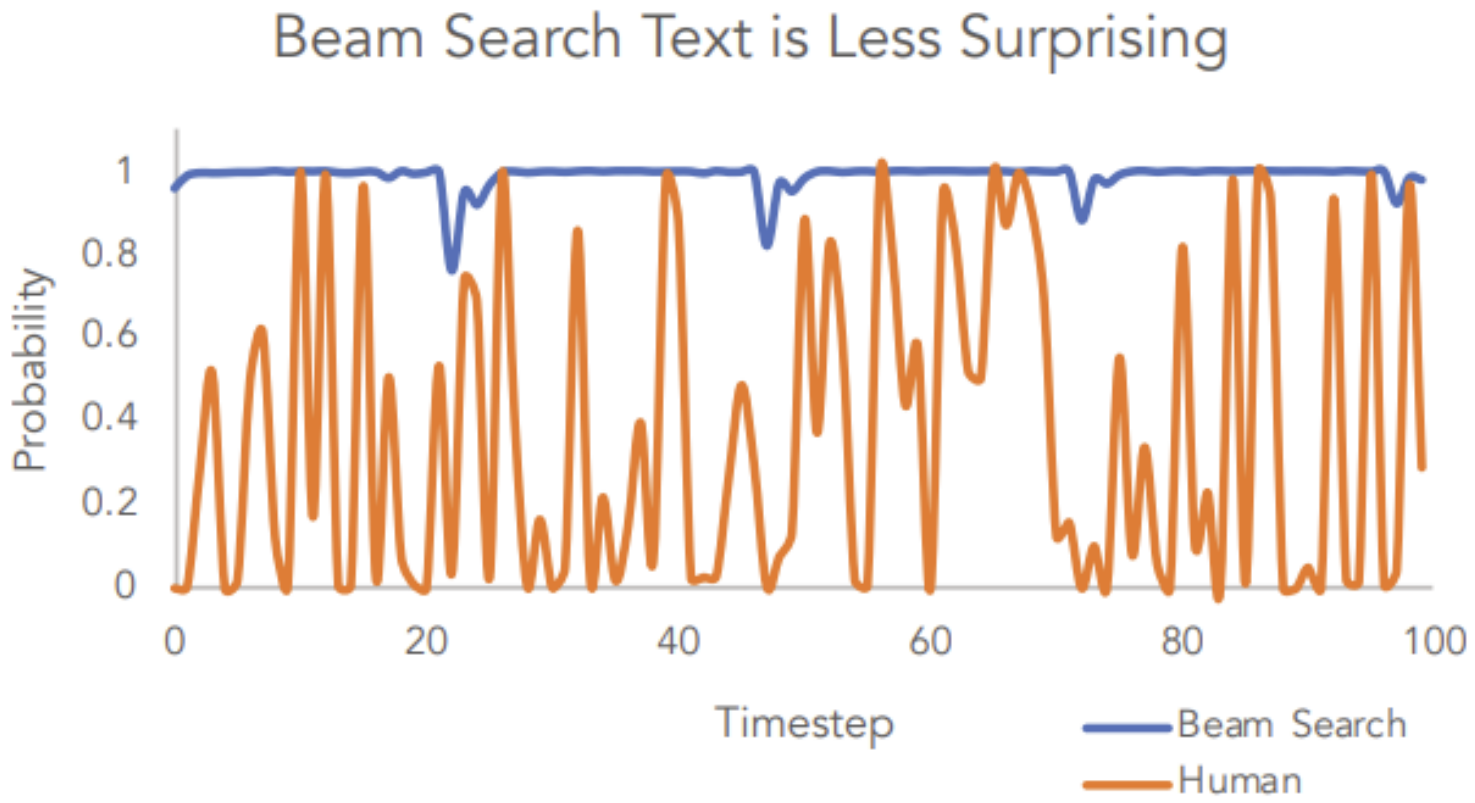
- Presence Penalty (>0)
  - New logit(token) = logit(token) – presence penalty*$\mathbb{I}_{\text{prev occurrence(token)}}$
- Frequency Penalty (>0)
  - New logit(token) = logit(token) - frequency penalty * #prev occurences(token)
- Repetition Penalty (>1)
  - New logit(token) = logit(token)/(repetition_penalty*$\mathbb{I}_{\text{prev occurrence(token)}}$)
  - New logit(token) = logit(token)*repetition_penalty*$\mathbb{I}_{\text{prev occurrence(token)}}$) if logit<0

Beam Search Problems

- Expensive!
  - O(beam size x vocabulary size x generation length)
- Lack of Diversity
- Repetition in Tokens

- Is generating highest probability output really desirable?

Do Greedy Methods Work for Open-Ended Generation?



A. Holtzman, et al. (2020): The Curious Case of Neural Text Degeneration

Greedy methods fail to capture the variance of human text distribution. How to fix?

Beam Search Problems

- Expensive!
  - O(beam size x vocabulary size x generation length)
- Lack of Diversity
- Repetition in Tokens

- Is generating highest probability output really desirable?
  - NO!
  - We need some combination of sampling and max!
    - (And we also need efficiency – so generally, no beam search!)

Lets Think About It

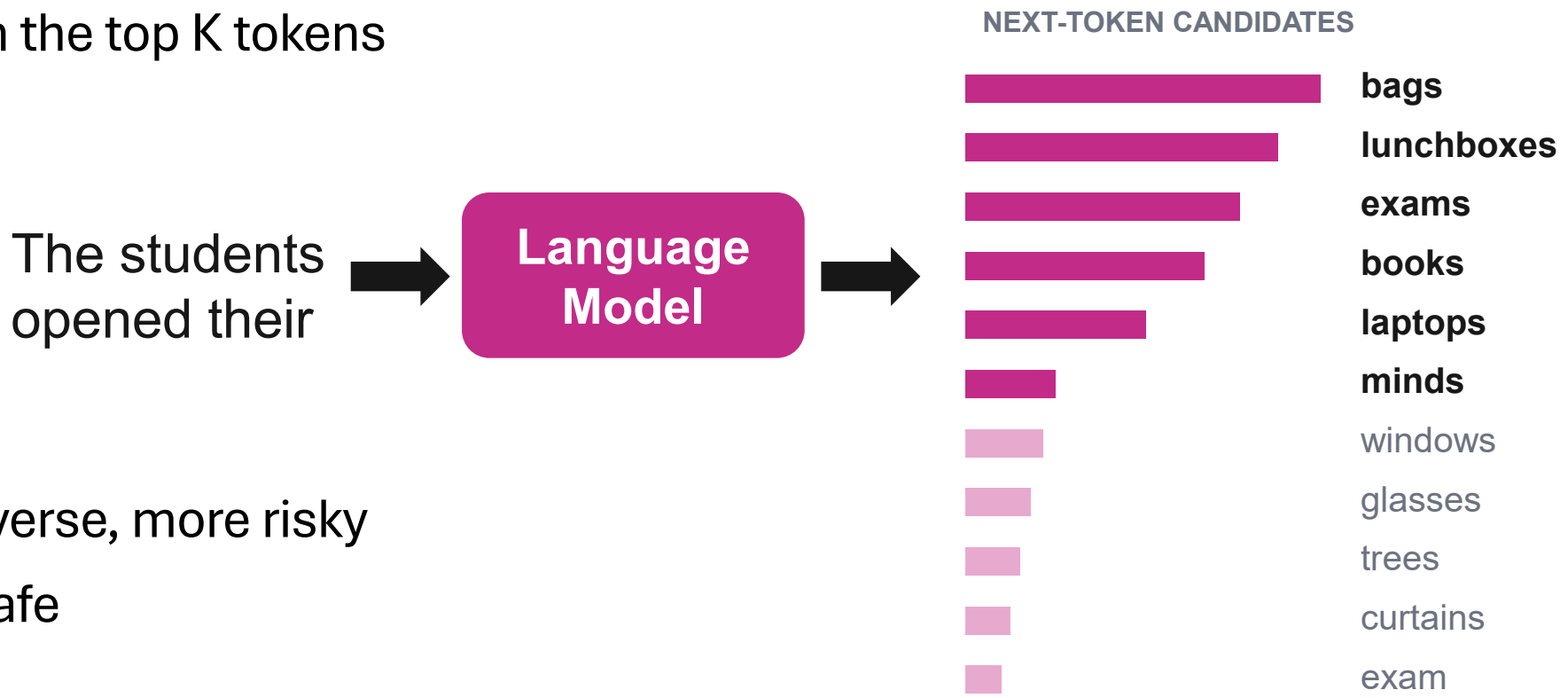
- Sampling
  - Increases diversity of output
  - But low probability tokens are likely really bad!

- Max
  - Reduces diversity
  - But highest probability (in general, top) tokens are good

- Lets mix the two ideas!

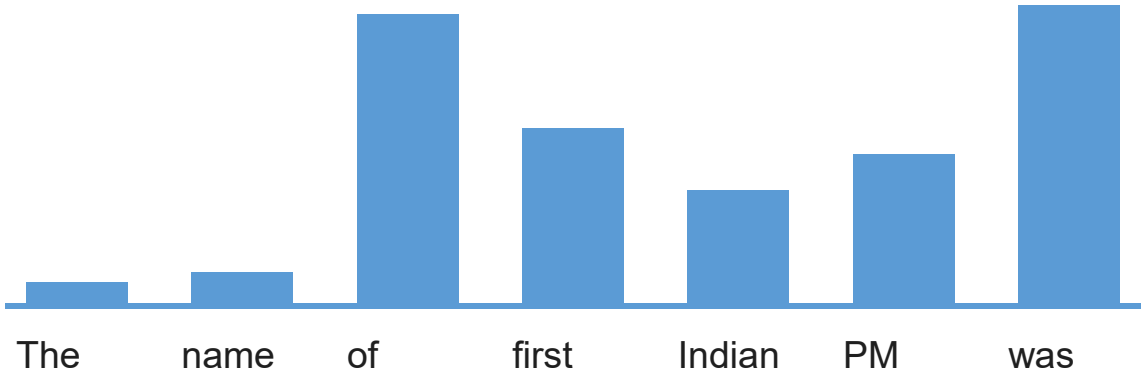
Top-K Sampling

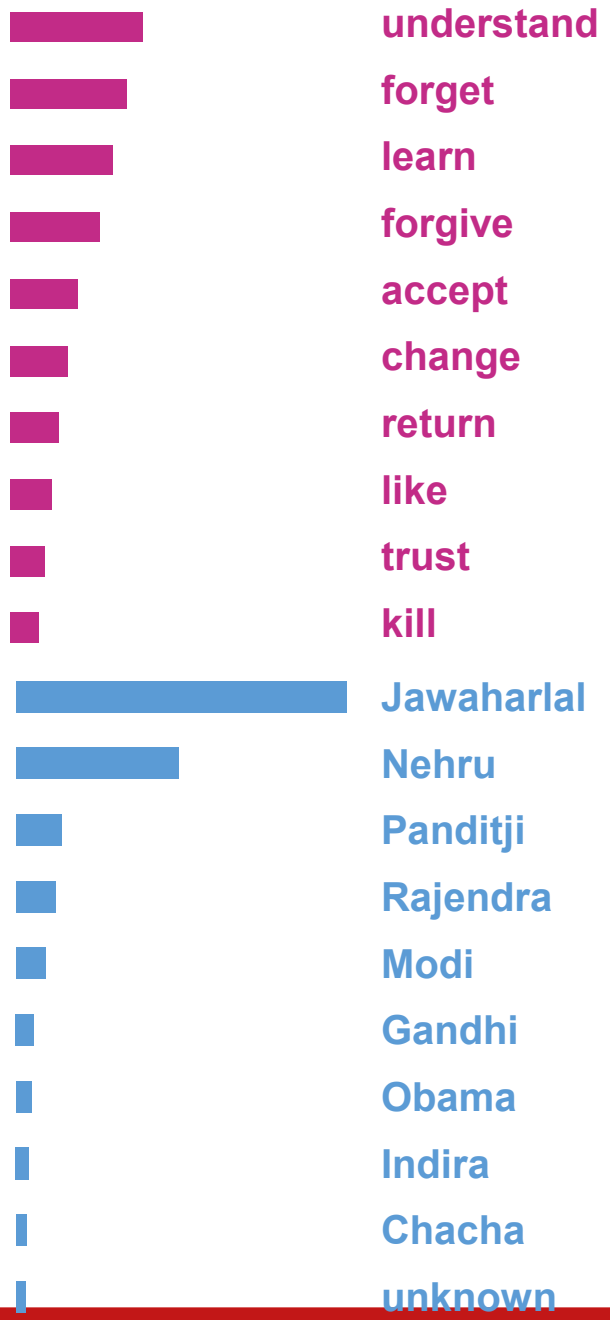
- Only Sample from the top K tokens in the distribution



- Higher K: more diverse, more risky
- Less K: generic, safe

Issues with Top-K Sampling





Top-p Sampling

- Top-k Sampling: The token distributions we sample from are dynamic

  When the distribution is flat, small K removes many viable options.

  When the distribution is peaked, large K allows too many bad options a chance to be selected.

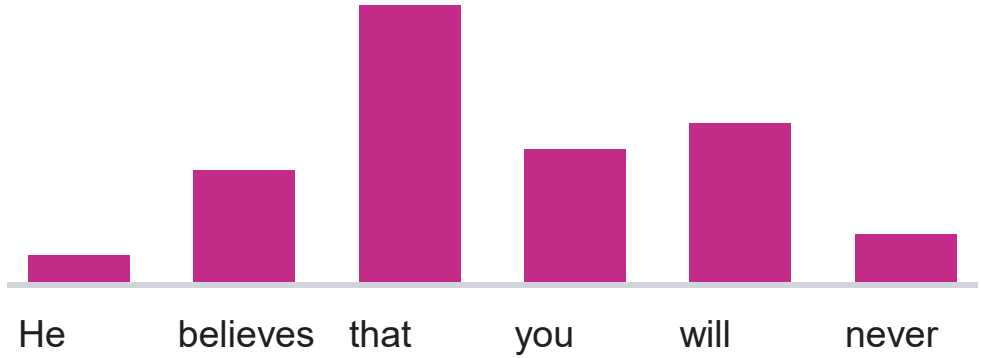
- Solution: Top-p (Nucleus) sampling (Holtzman et al., 2020)

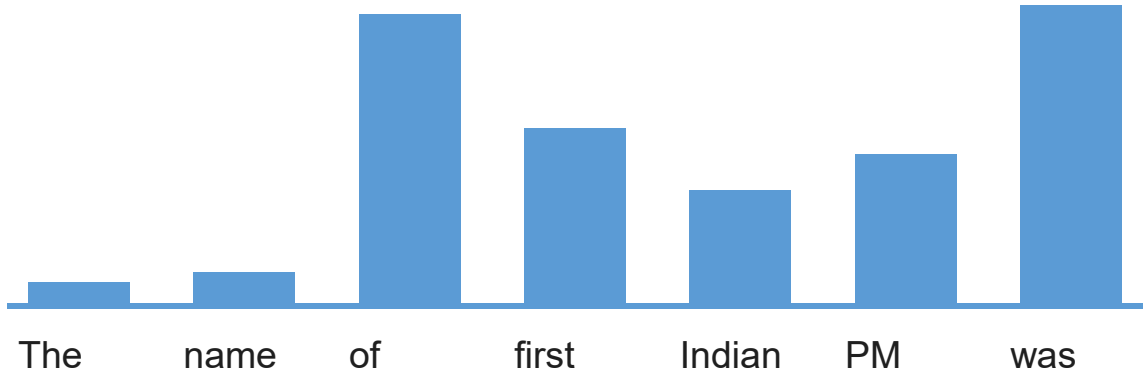
  Sample from all tokens in the top cumulative probability mass

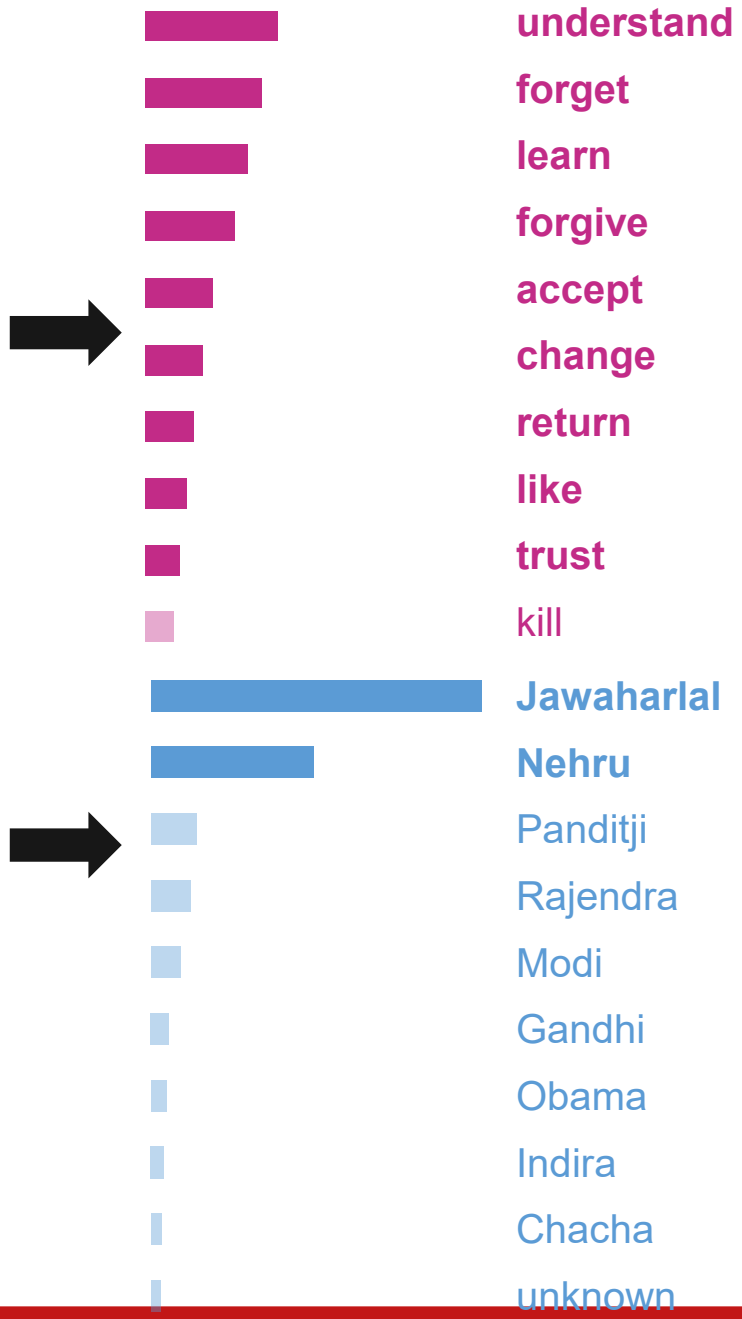
  (i.e., where mass is concentrated)

  Varies k according to the uniformity of P_t

Top-P Sampling (p=0.8)







Temperature Sampling

- Define a new temperature hyperparameter T that rebalances probability distribution
  - Raise the temperature T>1: Probability distribution becomes more uniform
  - Lower the temperature T<1: Probability distribution becomes more spiky

- Equations (let s_w be the logit for w in a probability distribution with context c)

$$P(w|c) = \frac{\exp(s_w/T)}{\sum_{w' \in V} \exp(s_{w'}/T)}$$

- Temperature is an independent hyperparameter that can be applied with any other generation algorithm (such as beam search, top-K, etc)

In Practice

- Goal: maintain fluency, consistency and creativity of generated text
- Common: use top-p sampling with temperature sampling and some repetition penalty

Method	Acts on	Effect	Purpose
Temperature	Token probabilities	Smoothly rescales all probabilities	Controls randomness
Top-p	Candidate token set	Truncates low-probability tokens	Limits unlikely outputs
Repetition penalty	Previously generated tokens	Downweights repeated tokens	Reduces repetition/loops